LOW LEVEL DESIGN / BUILD IT IN SIX PHASES

BookMyShow

One seat. One booking.

From a cinema catalog to expiring seat holds, payment strategies and concurrent bookings. Sketch the design, retype the code, and use the running examples to test your understanding.

1 **Venues and seats**
video reconstruction

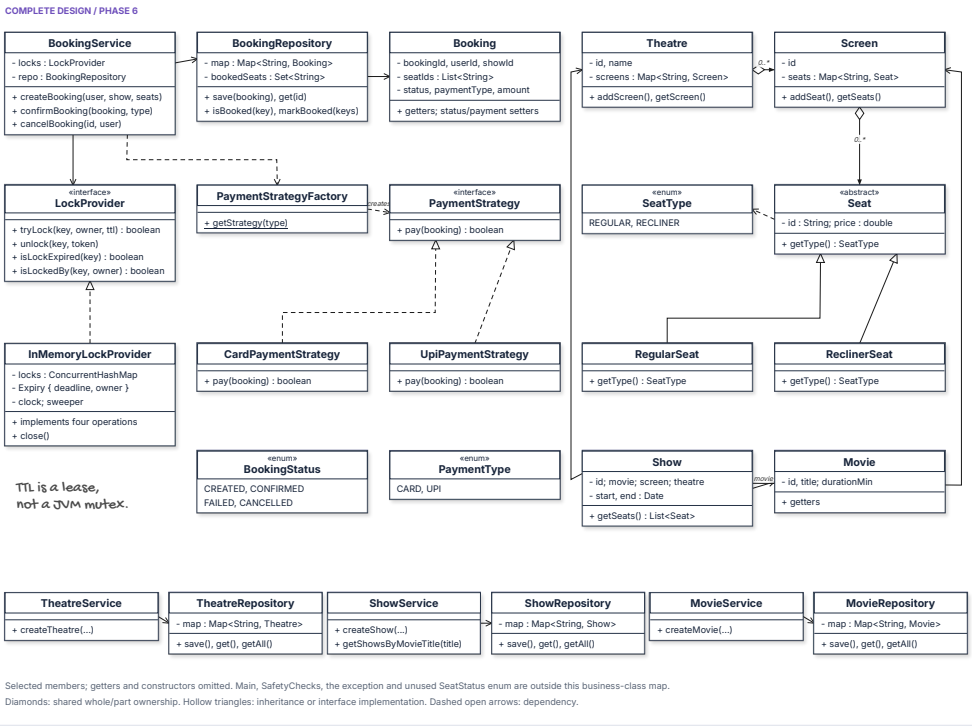
2 **Movies and shows**
video reconstruction

3 **Bookings and payments**
video reconstruction

4 **Expiring locks**
video reconstruction

5 **The booking flow**
video reconstruction

6 **Safety extensions**
safety extension



Source: Shubh Patel, *I Solved BookMyShow Seat Clash! Full LLD + Lock Expiry + Why Redis is Must* (45:26). Six independent Java 17 source folders · 16 video frames · growing UML · two sequence diagrams.
Provenance: reconstructed from the video; the author's public repository currently omits this case. Prepared 3 November 2026.

How to use these notes

> **Read → sketch → retype → run → explain.**

The study loop

1. Read one phase's goal and concepts.
2. Draw the classes from memory. Add only the new relationships.
3. Copy the previous phase into a practice folder and type the new code.
4. Run it, compare the output and inspect the diff.
5. Answer the checkpoint before moving on.

Each phase is complete. You never combine source directories. **NEW** tags identify added types; **UPDATED** tags and blue member highlights identify extension changes.

The PDF shows source read from the folders, with changed lines highlighted. Line numbers refer to the actual file. Some setup and test harness code is omitted from cards; the full files are included.

Contents

Understand the system	3
Destination and pattern toolkit	7
1. Venues and seats	9
2. Movies and shows	13
3. Bookings and payments	19
4. Expiring locks	25
5. The booking flow	30
6. Safety extensions	38
Complete code flow	48
Complete design	52
Notes on the reference code	53
Self-test and answer cues	55

Run on Windows

Open a phase folder and run `powershell -ExecutionPolicy Bypass -File .\run.ps1`. On macOS/Linux use `bash run.sh`. A JDK 17 or newer is required; no Maven, Gradle or Lombok.

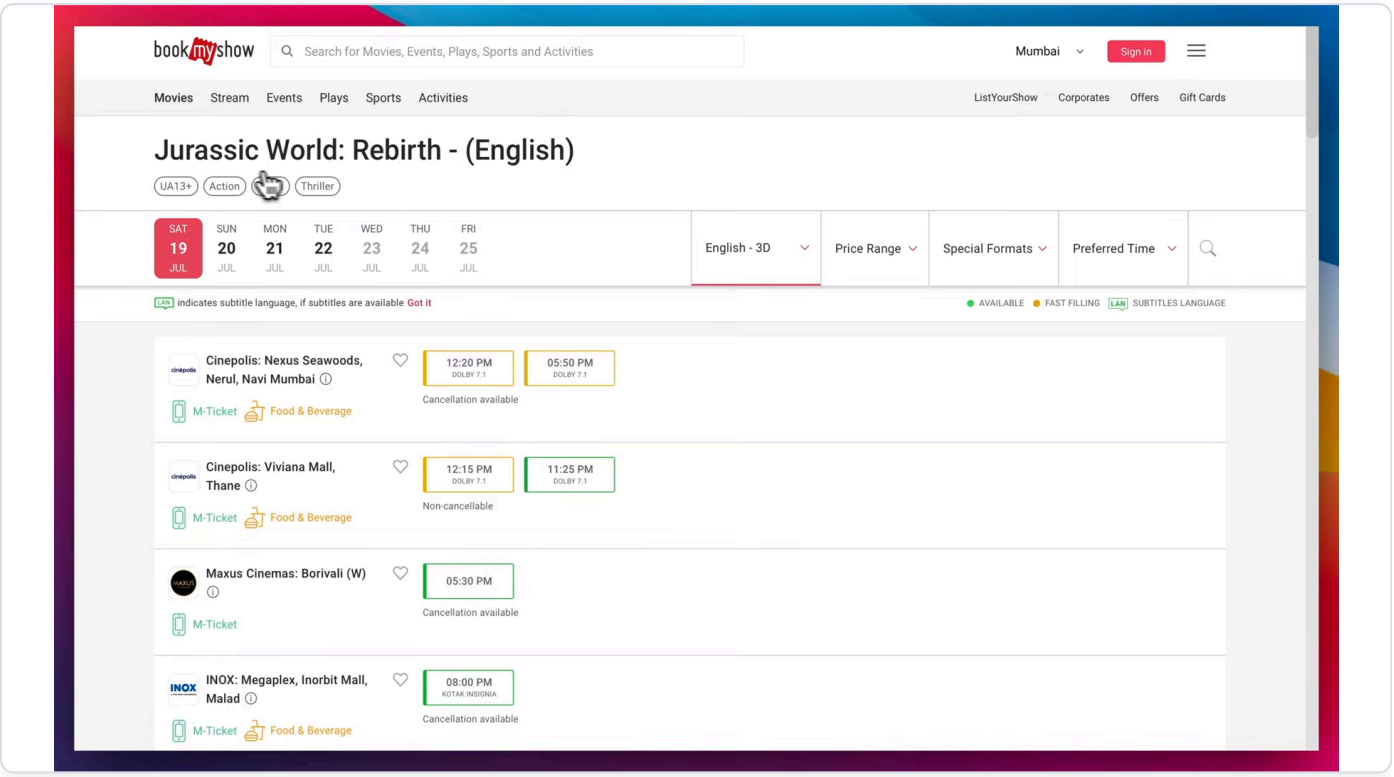
Two scopes to keep separate

Phases 1–5 reconstruct the teaching implementation, including its visible gaps. Phase 6 changes the behavior deliberately. None of the payment strategies contacts a real payment provider.

Output was captured from actual runs. The machine's JDK 22 compiled every phase with `--release 17`; execution was on JDK 22. The five-second expiry examples use an injected clock so they run immediately and reproducibly.

Step 0 · Understand the system

A movie is a title; a show is one screening of that title at one time and place. The user first picks a movie, then a show, then seats for that show. The hard part begins when two users want the same seat.



01:00 · The website demo lists showtimes under theatres.

Catalog path

Movie title → matching shows → theatre and screen → physical seats. These objects tell us what exists.

Booking path

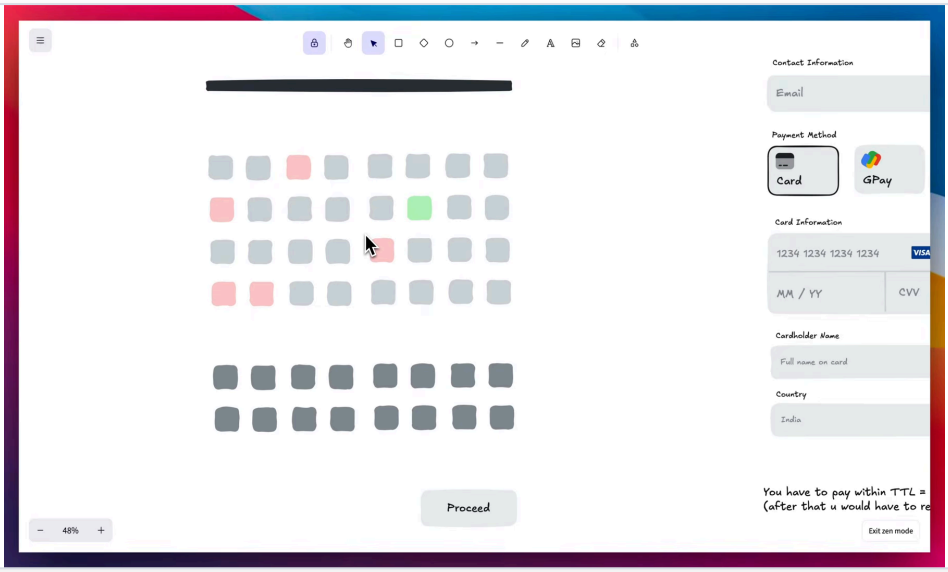
Select seats → acquire temporary holds → attempt payment → confirm or release. These operations decide who can occupy the seats.

Why show ID belongs in the lock key

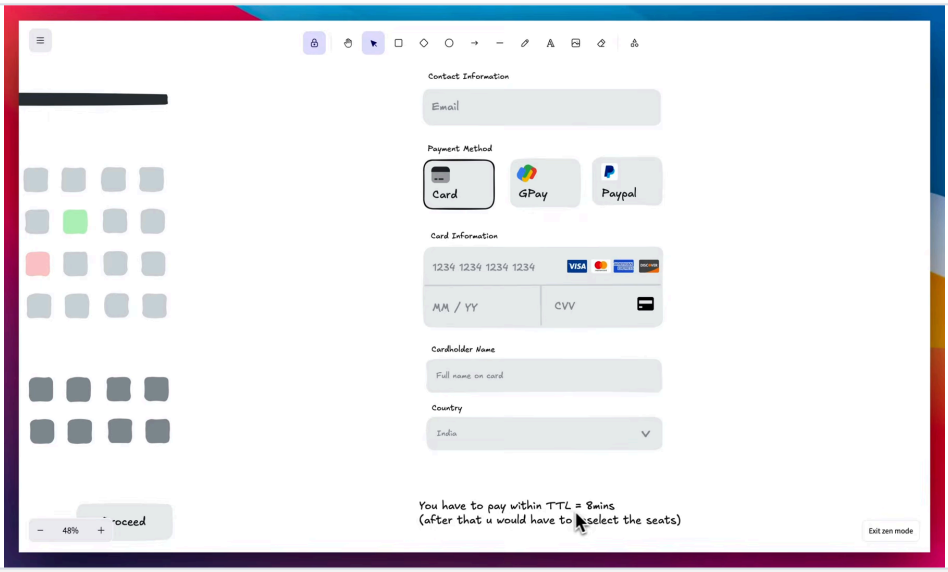
The same physical seat can be sold for the 18:30 show and the 21:00 show. `showId + ":" + seatId` scopes the claim to one performance. A key containing only the seat ID would block other shows unnecessarily.

A seat is a chair. A show-seat is something you can sell.

Selection is not a reservation



02:00 · The teaching board distinguishes selected, booked and available seats.

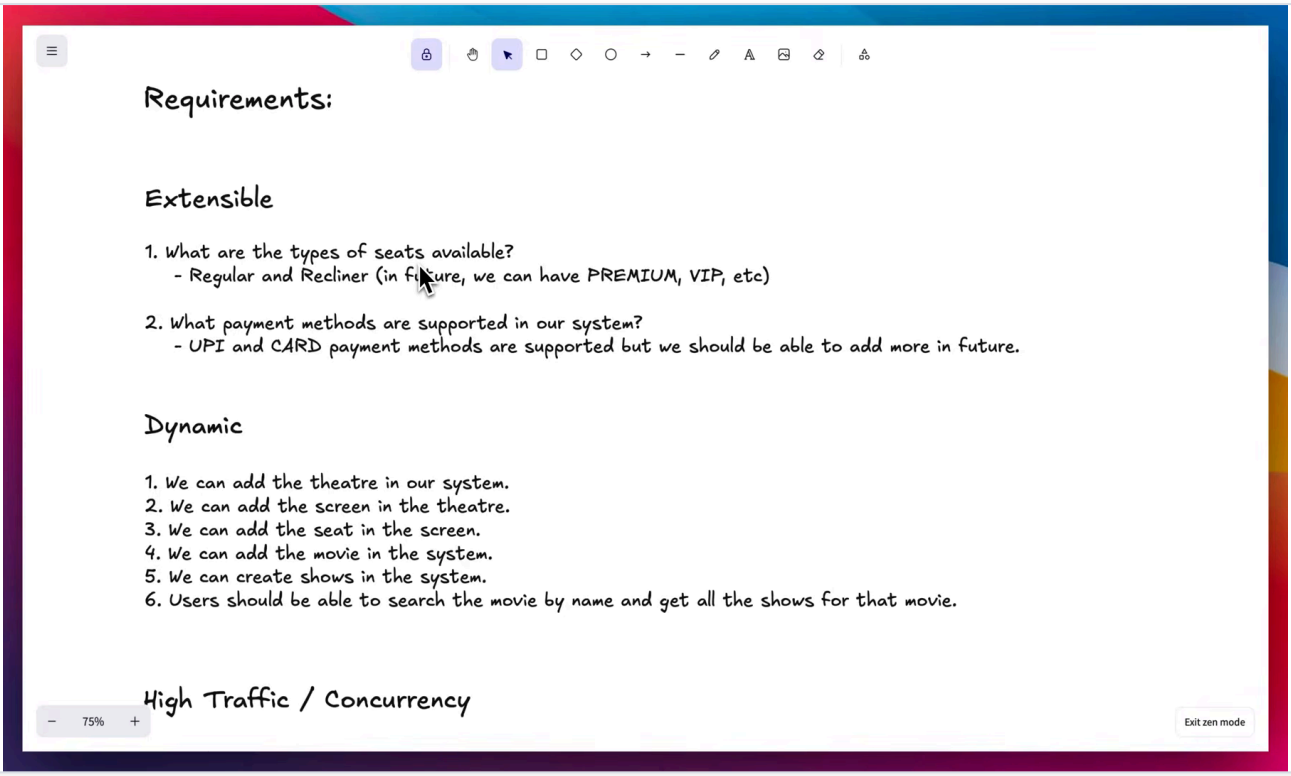


03:00 · The presenter describes an eight-minute payment window.

> **What the timer means**
>
> In this video, clicking Proceed creates the hold. Clicking Pay attempts confirmation. The presenter uses roughly eight minutes for the product explanation, then five seconds in the Java demo. This is the video's example, not a claim about the current BookMyShow policy.

At expiry: another user may acquire the seat. The original browser can still show a Pay button, so the server must re-check the hold's validity and ownership. A stale page is not proof of a valid reservation.

Requirements before classes



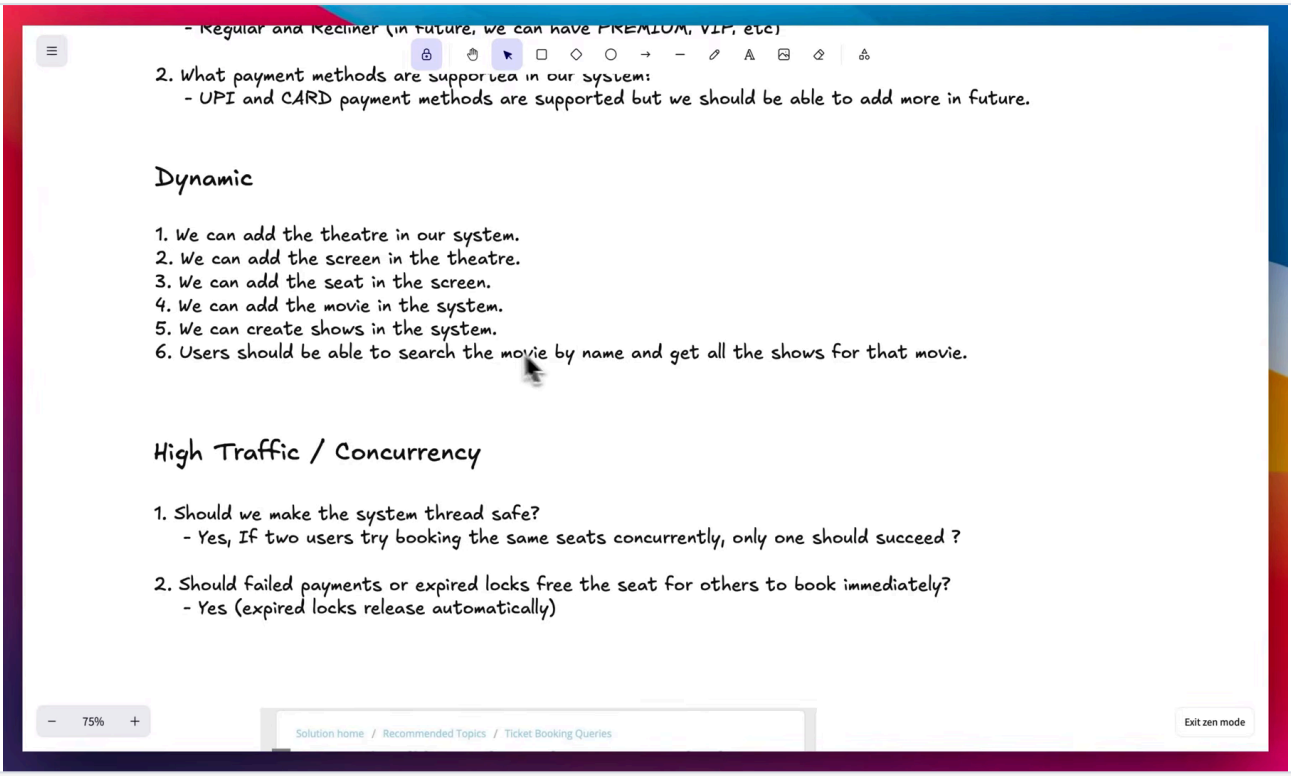
04:40 · Extensible seat categories, payment methods and dynamic catalog operations.

Requirement	Design consequence
Regular and recliner seats; more categories later	Abstract Seat with concrete subtypes. Price belongs to the seat in this teaching model.
CARD and UPI; more methods later	PaymentStrategy implementations selected by a centralized Simple Factory.
Add theatres, screens, seats, movies and shows	Catalog services apply business operations; repositories store and retrieve objects.
Find shows by movie title	ShowService filters the repository; the first version scans all shows.
Only one live claim per show-seat	An atomic per-key acquisition operation with owner and expiry.
Abandoned or failed attempts free seats	Deadline checks, safe release and rollback; some of these are missing from the shown code.

Ask the interviewer

Is a seat group all-or-nothing? Does payment start before the deadline or finish before it? Are refunds and cancellations required? Is this one process or several servers? The answers determine the consistency boundary.

The worked example: one contested seat



05:40 · The requirements explicitly call for one winner and eventual release.

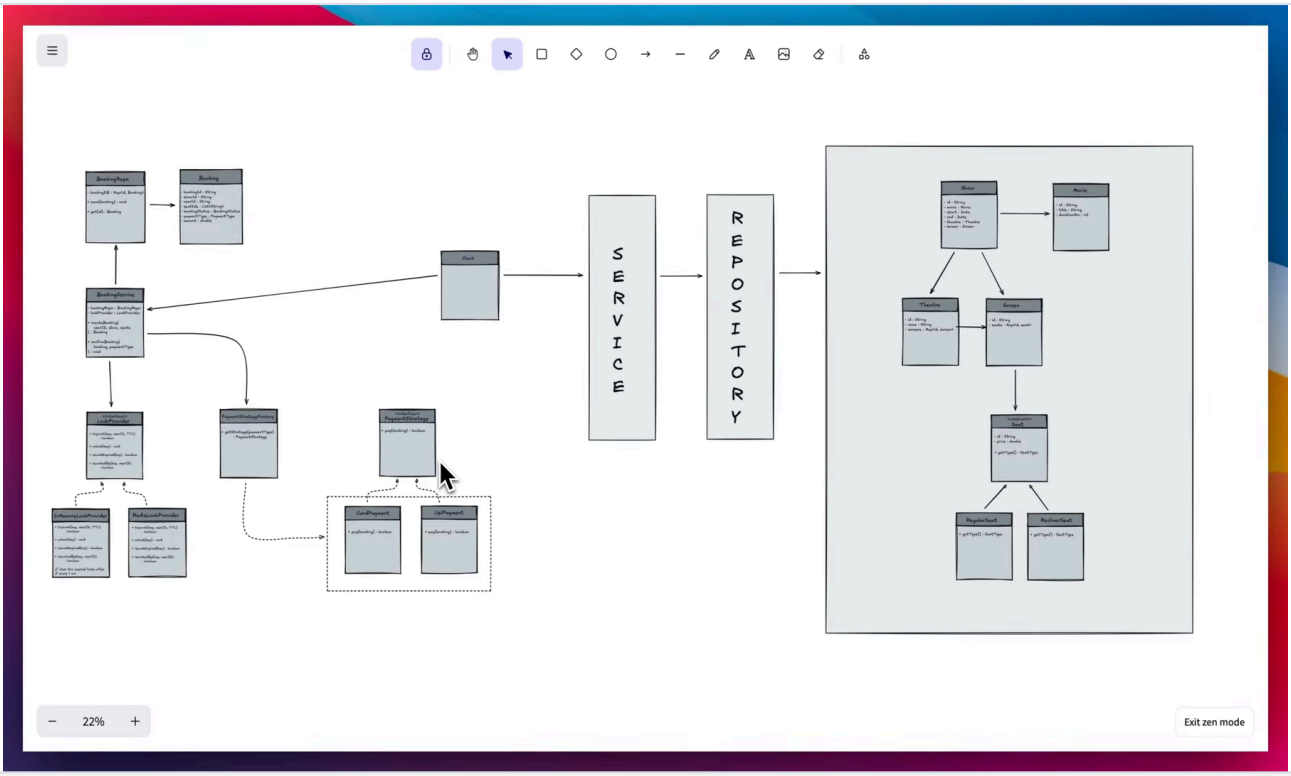
Time / request	Expected result
user2 asks for [s1-3, s1-4]; user3 asks for [s1-4, s1-5]	Only one booking can hold s1-4. The losing request should leave no partial hold.
t = 0: user4 holds [s1-6, s1-7], TTL = 5 s	Booking is CREATED. No money has been paid.
t = 6 s: user5 asks for the same seats	The old deadline has passed; user5 can acquire fresh holds.
user4 now clicks Pay	Reject before invoking payment: user4 no longer has a valid hold.
user5 pays successfully	Both seats become permanently unavailable for this show.

State the invariant

For each (showId, seatId), at most one valid hold or confirmed allocation may authorize a booking. A booking group should either obtain all requested seats or leave none held by that attempt.

The last sentence is a requirement, not something automatically supplied by a concurrent map. Phase 5 exposes the difference; Phase 6 enforces it inside one shared repository monitor.

The destination · Read the whiteboard



17:00 · Full-board overview: booking orchestration on the left, catalog and entities on the right.

Two flows

Create: acquire locks → calculate amount → save CREATED booking.

Confirm: inspect state → validate holds → choose payment strategy → pay → release temporary locks → change status.

Do not infer missing behavior

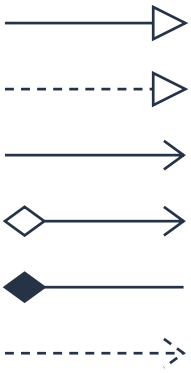
The whiteboard includes a RedisLockProvider, but no Redis implementation is demonstrated. A SeatStatus enum appears in code, but it is not connected to a permanent allocation store in the shown flow.

Reconstruction, not an official checkout

The author's public repository, checked at commit `fd753ebd1fc03b6a499c8671151f62ed7dc13c05`, has no BookMyShow package. The video description says notes/code will be provided later. Names and behavior here follow visible source and narration; unseen bodies are reconstructed and adaptations are listed at the end.

The overview is useful for orientation. Close-up frames and the recreated vector diagrams supply the detail. The two older study-pack folders named by the skill were not present on this Windows machine; the supplied Splitwise build scripts and README were used as the format reference.

Arrow legend and pattern toolkit



Inheritance. Solid line + hollow triangle: subclass → superclass.

Realization. Dashed line + hollow triangle: implementation → interface.

Association. Solid open arrow: one object knows or uses another.

Aggregation. Hollow diamond at the whole: shared whole/part relationship.

Composition. Filled diamond: exclusive lifecycle ownership; not implied by every field.

Dependency. Dashed open arrow: dependency, such as a factory selecting a strategy.

`«interface»` and `«enum»` identify the type. Underlined methods are static. The class maps show selected members, not every constructor or getter. A method's return enum value is behavior; its UML return type is still the enum type.

Strategy

CARD and UPI implement the same pay(Booking) contract. The service does not contain gateway-specific algorithms.

Simple Factory

One switch maps PaymentType to a concrete strategy. This is a Simple Factory, not the GoF Factory Method pattern.

Repository

Save/get operations hide the in-memory collection. These repositories are concrete classes; a separate interface would be a further refactor.

Dependency injection

Constructors receive repositories and the LockProvider. Main builds and connects the object graph.

Polymorphism

RegularSeat and ReclinerSeat extend Seat. Shared identity and price stay in the base class.

Lease

A claim has an owner and a deadline. Expiry provides liveness; ownership checks protect against stale users.

A useful SOLID correction

A factory localizes selection logic, but an if/else in a client is not automatically a SOLID violation. Adding a strategy still changes the enum and factory; most booking logic can remain unchanged.

PHASE

1

Phase 1 · Venues and seats

entities · inheritance · aggregation

Model the physical cinema without confusing a chair with its availability for a particular show.

`Phase-1-Venues-and-Seats/`

`SeatType` `ReclinerSeat` `RegularSeat` `Screen` `Seat` `Theatre`

Separate identity from behavior

`Seat` contains an ID and price, then requires subclasses to implement `getType()`. Regular and recliner seats differ by type; both can be stored in one `Map<String, Seat>`.

Build the ownership chain

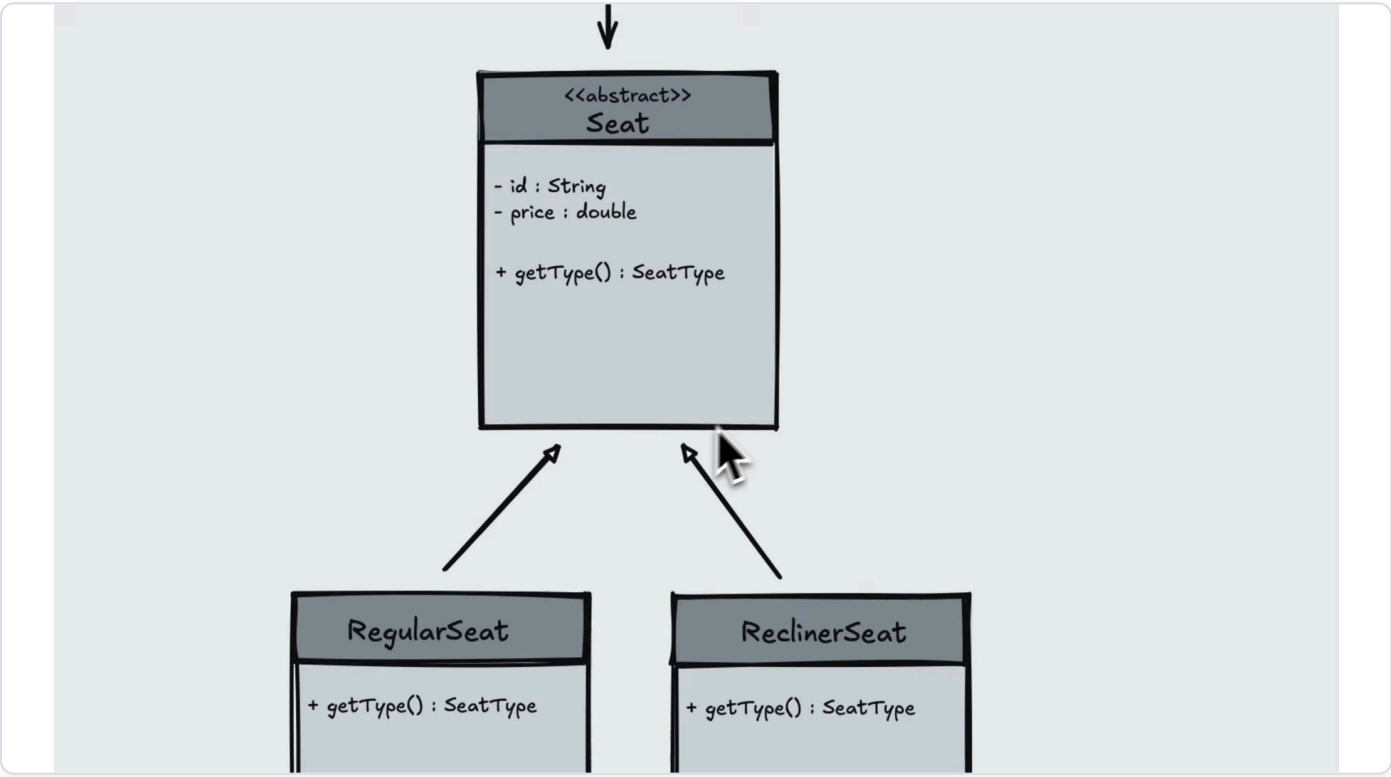
A Theatre stores Screens. A Screen stores Seats. The objects are created outside these containers and then added, so the diagrams use shared aggregation rather than asserting strict lifecycle ownership.

Why maps?

The video chooses maps to locate objects by ID. Theatre.getScreen is a direct lookup. Screen.getSeats returns a copied list; the screen's map stays private.

Price is supplied, not derived

The constructors accept a price. The demo supplies 150 for regular and 300 for recliner. The class name alone does not hard-code a price.



09:30 · Seat inheritance in the whiteboard design.

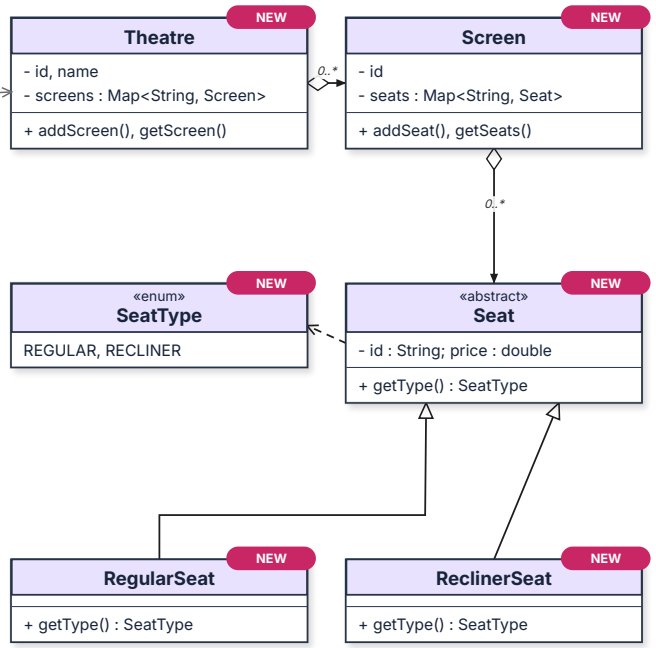
There is deliberately no global `Seat.booked` flag. A chair can appear in many shows, and each show has its own booking inventory. We will build that distinction into lock keys.

Phase 1 · The growing class diagram

PHASE 1 / GROWING CLASS MAP

Start with physical seats. Availability belongs to a show, not to the chair itself.

The empty space will become bookings, payments and locks.



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.

Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 1 · Code lab 1

Read from `Phase-1-Venues-and-Seats/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

SeatType.java NEW

```
3  public enum SeatType { REGULAR, RECLINER }
```

ReclinerSeat.java NEW

```
5  public class ReclinerSeat extends Seat {
6      public ReclinerSeat(String id, double price) { super(id, price); }
7      @Override
8      public SeatType getType() { return SeatType.RECLINER; }
9  }
```

RegularSeat.java NEW

```
5  public class RegularSeat extends Seat {
6      public RegularSeat(String id, double price) { super(id, price); }
7      @Override
8      public SeatType getType() { return SeatType.REGULAR; }
9  }
```

Screen.java NEW

```
5  public class Screen {
6      private final String id;
7      private final Map<String, Seat> seats = new LinkedHashMap<>();
8      public Screen(String id) { this.id = id; }
9      public String getId() { return id; }
10     public void addSeat(Seat seat) { seats.put(seat.getId(), seat); }
11     public List<Seat> getSeats() { return new ArrayList<>(seats.values()); }
12 }
```

Seat.java NEW

```
5  public abstract class Seat {
6      private final String id;
7      private final double price;
8      protected Seat(String id, double price) {
9          this.id = id;
10         this.price = price;
11     }
12     public String getId() { return id; }
13     public double getPrice() { return price; }
14     public abstract SeatType getType();
15 }
```

Theatre.java NEW

```
5  public class Theatre {
6      private final String id;
7      private final String name;
8      private final Map<String, Screen> screens = new LinkedHashMap<>();
9      public Theatre(String id, String name) {
10         this.id = id;
11         this.name = name;
12     }
13     public String getId() { return id; }
14     public String getName() { return name; }
15     public void addScreen(Screen screen) {
16         screens.put(screen.getId(), screen);
17     }
18     public Screen getScreen(String id) { return screens.get(id); }
19 }
```

Phase 1 · Run it and check yourself

```
Phase-1-Venues-and-Seats / actual output
PVR Phoenix / s1
s1-1 REGULAR INR 150.00
s1-9 RECLINER INR 300.00
Physical seats have no show-specific status.
```

Answer cue

A copied list protects collection membership, not the mutable state of its elements. This pack's Seat identity and price are final. A VIP subtype also needs a SeatType entry; callers using Seat need not change.

Checkpoint

1. What goes wrong if Seat has one global booked flag?
2. Why can both concrete seat types live in the same map?
3. Does returning a copied list make the Seat objects immutable?
4. What must change to introduce VIP seats?

Retype drill · Add one more seat type

Start with an empty sheet and draw the Theatre → Screen → Seat relationships. Add a VIPSeat subtype, decide its SeatType value, and pass its price through the constructor. Add it to the existing Screen without changing the loop that prints Seat objects.

Predict before running	What to look for
Add the same seat ID twice	The map replaces the old entry; duplicate prevention is not implemented.
Call getSeats(), then clear that returned list	The Screen still holds its original seats because the list is a copy.
Use one Seat in two different shows later	Its identity can be shared; availability must still be tracked per show.

Explain the tradeoff

The two subtypes currently differ only in getType. An enum-backed value object could be simpler today. We retain the inheritance design to match the video and leave room for seat-specific behavior.

PHASE 2

Phase 2 · Movies and shows

Repository · service layer · search

Let an admin build the catalog, then search shows by movie title.

`Phase-2-Movies-and-Shows/`

`Movie` `Show` `MovieRepository` `ShowRepository` `TheatreRepository` `MovieService` `ShowService` `TheatreService`

Movie versus Show

Movie carries title and duration. Show connects that movie to a theatre, screen, start and end time. Several Show objects can refer to the same Movie.

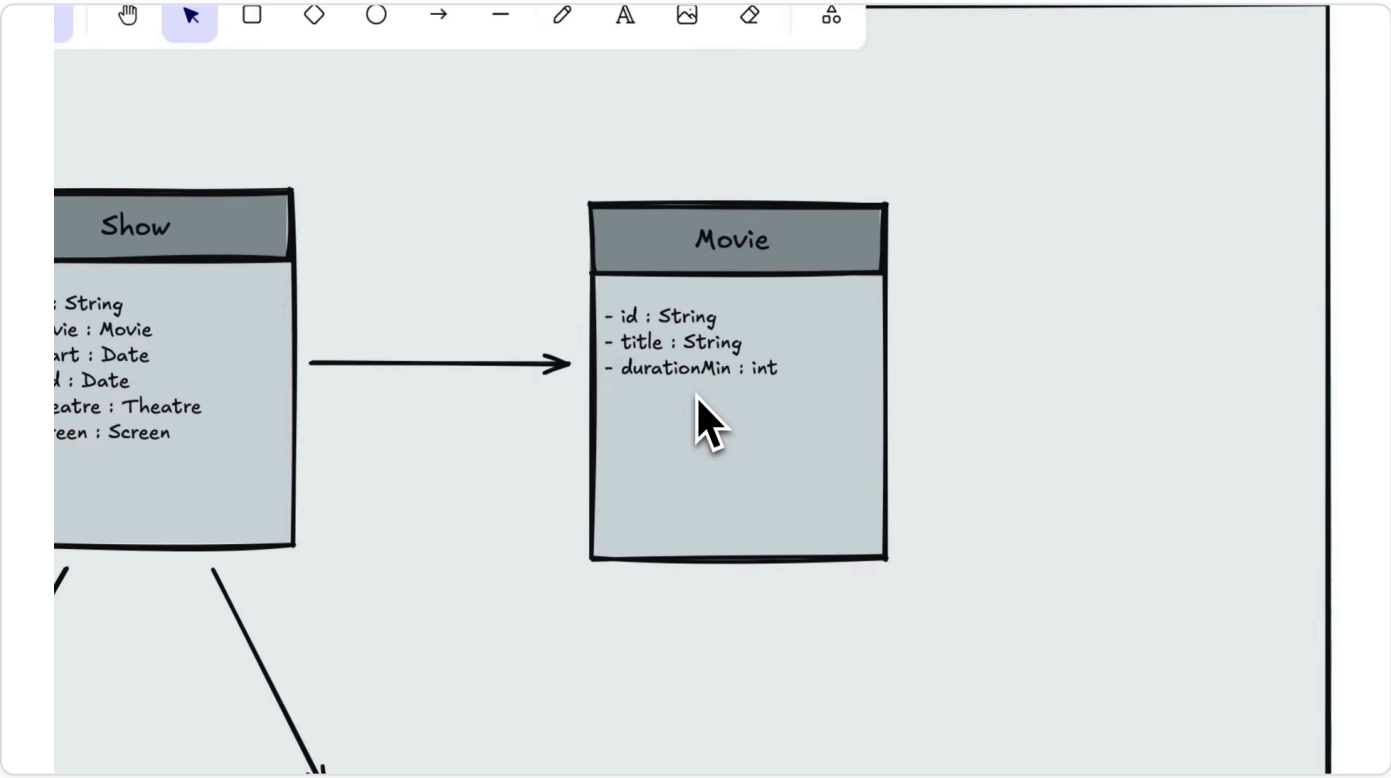
The repository stores objects; the service creates or finds them. ShowService resolves movie and theatre IDs before constructing a Show. Main is the composition root that wires repositories into services.

Why not put search in Main?

The use case belongs in ShowService. Main should describe the example, not know how shows are stored or filtered.

Keep the scope honest

The catalog assumes valid IDs, setup before booking traffic, and no overlapping-show validation. Those are additional requirements, not guarantees of these small CRUD wrappers.



10:00 · Movie is shared data; Show adds the screening's time and location. The left edge is cropped in the source video.

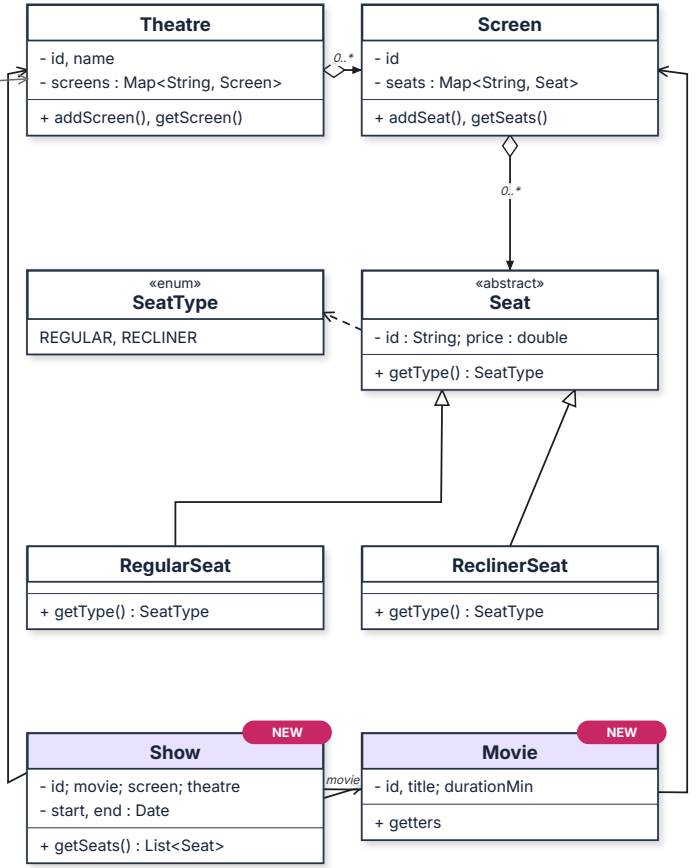
Search: the code scans all shows and compares movie titles case-insensitively. With S shows, that is O(S) visits plus title-comparison cost. A title index is a possible later optimization, not needed for this example.

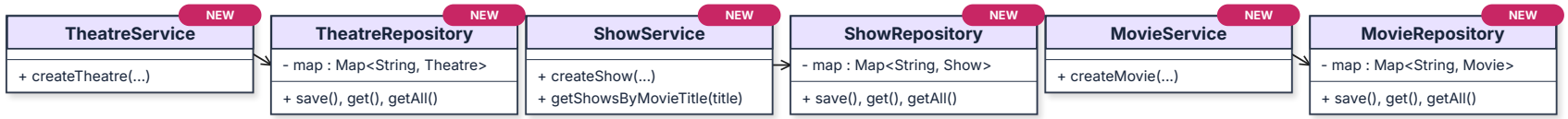
Phase 2 · The growing class diagram

PHASE 2 / GROWING CLASS MAP

Start with physical seats. Availability belongs to a show, not to the chair itself.

The empty space will become bookings, payments and locks.





Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.
Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 2 · Code lab 1

Read from `Phase-2-Movies-and-Shows/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

Movie.java NEW

```java
public class Movie {
    private final String id;
    private final String title;
    private final int durationMin;
    public Movie(String id, String title, int durationMin) {
        this.id = id;
        this.title = title;
        this.durationMin = durationMin;
    }
    public String getId() { return id; }
    public String getTitle() { return title; }
    public int getDurationMin() { return durationMin; }
}
```

Show.java NEW

```java
public class Show {
    private final String id;
    private final Movie movie;
    private final Date start;
    private final Date end;
    private final Screen screen;
    private final Theatre theatre;
    public Show(String id, Movie movie, Date start,
                Screen screen, Theatre theatre) {
        this.id = id;
        this.movie = movie;
        this.start = new Date(start.getTime());
        this.end = new Date(start.getTime()
            + movie.getDurationMin() * 60_000L);
        this.screen = screen;
        this.theatre = theatre;
    }
    public String getId() { return id; }
    public Movie getMovie() { return movie; }
    public Date getStart() { return new Date(start.getTime()); }
    public Date getEnd() { return new Date(end.getTime()); }
    public Screen getScreen() { return screen; }
    public Theatre getTheatre() { return theatre; }
    public List<Seat> getSeats() { return screen.getSeats(); }
}
```

MovieRepository.java NEW

```java
public class MovieRepository {
    private final Map<String, Movie> map = new LinkedHashMap<>();
    public void save(Movie value) { map.put(value.getId(), value); }
    public Movie get(String id) { return map.get(id); }
    public List<Movie> getAll() {
        return new ArrayList<>(map.values());
    }
}
```

Phase 2 · Code lab 2

Read from `Phase-2-Movies-and-Shows/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

ShowRepository.java NEW

```
 6 public class ShowRepository {
 7     private final Map<String, Show> map = new LinkedHashMap<>();
 8     public void save(Show value) { map.put(value.getId(), value); }
 9     public Show get(String id) { return map.get(id); }
10     public List<Show> getAll() {
11         return new ArrayList<>(map.values());
12     }
13 }
```

TheatreRepository.java NEW

```
 6 public class TheatreRepository {
 7     private final Map<String, Theatre> map = new LinkedHashMap<>();
 8     public void save(Theatre value) { map.put(value.getId(), value); }
 9     public Theatre get(String id) { return map.get(id); }
10     public List<Theatre> getAll() {
11         return new ArrayList<>(map.values());
12     }
13 }
```

MovieService.java NEW

```
 6 public class MovieService {
 7     private final MovieRepository movieRepo;
 8     public MovieService(MovieRepository movieRepo) {
 9         this.movieRepo = movieRepo;
10     }
11     public Movie createMovie(String id, String title, int durationMin) {
12         Movie movie = new Movie(id, title, durationMin);
13         movieRepo.save(movie);
14         return movie;
15     }
16     public Movie getMovie(String id) { return movieRepo.get(id); }
17 }
```

Phase 2 · Code lab 3

Read from `Phase-2-Movies-and-Shows/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

ShowService.java NEW

```java
public class ShowService {
    private final ShowRepository showRepo;
    private final TheatreRepository theatreRepo;
    private final MovieRepository movieRepo;
    public ShowService(ShowRepository showRepo,
                       TheatreRepository theatreRepo,
                       MovieRepository movieRepo) {
        this.showRepo = showRepo;
        this.theatreRepo = theatreRepo;
        this.movieRepo = movieRepo;
    }
    public Show createShow(String showId, String movieId,
                           String theatreId, String screenId,
                           Date start) {
        Theatre theatre = theatreRepo.get(theatreId);
        Show show = new Show(showId, movieRepo.get(movieId), start,
            theatre.getScreen(screenId), theatre);
        showRepo.save(show);
        return show;
    }
    public Show getShow(String id) { return showRepo.get(id); }
    public List<Show> getShowsByMovieTitle(String title) {
        return showRepo.getAll().stream()
            .filter(s -> s.getMovie().getTitle().equalsIgnoreCase(title))
            .toList();
    }
}
```

TheatreService.java NEW

```java
public class TheatreService {
    private final TheatreRepository theatreRepo;
    public TheatreService(TheatreRepository theatreRepo) {
        this.theatreRepo = theatreRepo;
    }
    public Theatre createTheatre(String id, String name) {
        Theatre theatre = new Theatre(id, name);
        theatreRepo.save(theatre);
        return theatre;
    }
    public void addScreen(String theatreId, Screen screen) {
        theatreRepo.get(theatreId).addScreen(screen);
    }
    public void addSeat(String theatreId, String screenId, Seat seat) {
        theatreRepo.get(theatreId).getScreen(screenId).addSeat(seat);
    }
}
```

Phase 2 · Run it and check yourself

```
Phase-2-Movies-and-Shows / actual output
Search: interstellar
show1 | Interstellar | PVR Phoenix | s1 | 18:30-20:30 IST
Seats: 10
No matches: 0
```

Example setup

18:12 · Interstellar, 120 minutes, screen s1 in PVR Phoenix, starting at 18:30. The pack fixes the date and prints IST explicitly to make output independent of the machine's timezone.

Checkpoint

1. Why is Show a separate class from Movie?
2. Where would you reject a show with a missing screen?
3. What does the repository hide, and what does it not hide?
4. How would you avoid scanning every show for each title search?

Trace the catalog creation

Caller	Operation / result
Main → MovieService	Create Interstellar with durationMin = 120.
MovieService → MovieRepository	Save under m1.
Main → ShowService	Resolve m1, t1 and s1; create show1.
Show constructor	Start 18:30; end = start + 120 minutes.
ShowService → ShowRepository	Store show1; title search later filters all stored shows.

Retype drill

Create a second show of the same movie at 21:00 with a different show ID. Search for INTERSTELLAR and predict the number of returned shows. Then search for an unrelated title. Neither query should mutate the catalog.

PHASE

3

Phase 3 · Bookings and payments

Strategy · Simple Factory · state

Represent a booking and choose a payment algorithm through one interface.

`Phase-3-Bookings-and-Payments/`

`BookingStatus` `PaymentType` `SeatStatus` `PaymentStrategyFactory` `Booking` `BookingRepository` `CardPaymentStrategy` `PaymentStrategy` `UpiPaymentStrategy`

A booking records an attempt

Booking holds user/show IDs, selected seat IDs, amount, status and payment type. It begins in CREATED. That is a pending checkout, not proof of a completed sale.

Strategy owns the algorithm

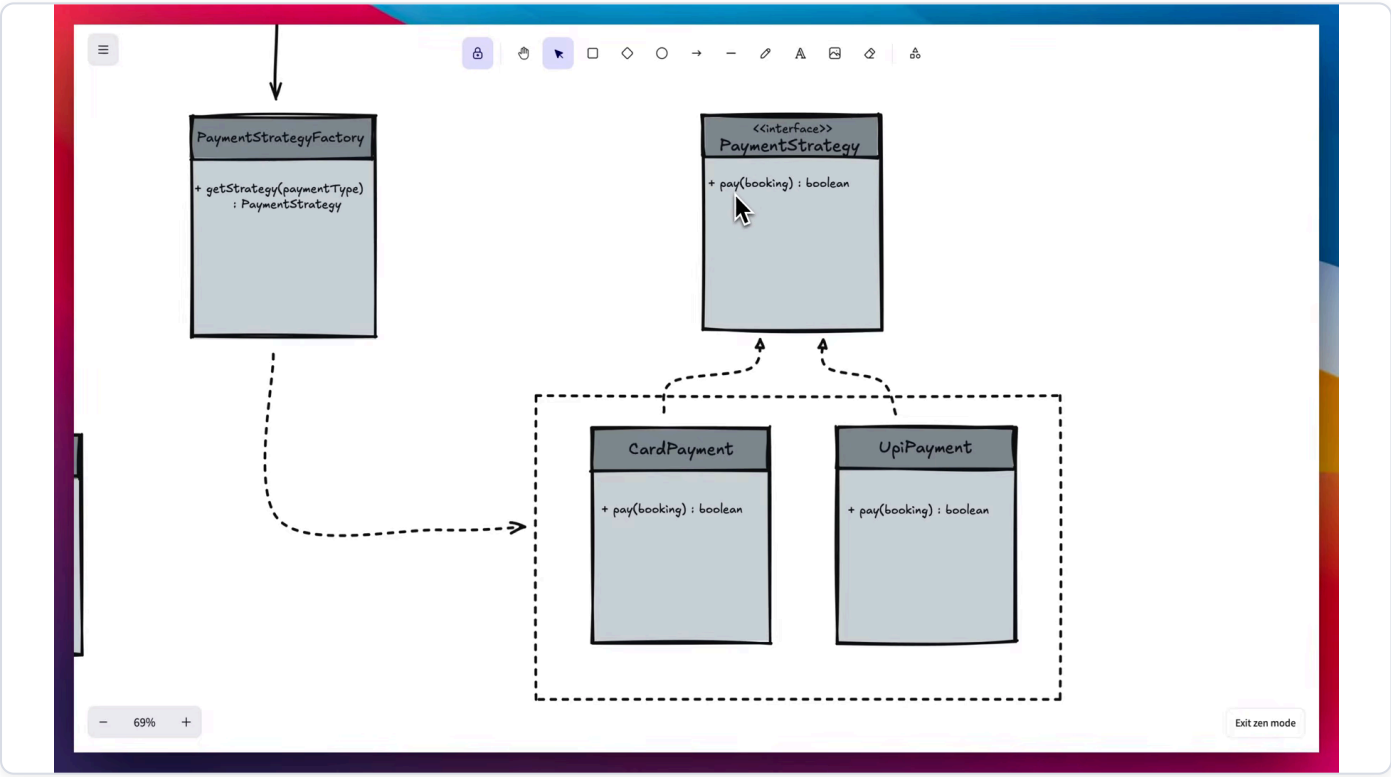
CardPaymentStrategy and UpiPaymentStrategy implement `boolean pay(Booking)`. They log and return true. PaymentStrategyFactory chooses one from PaymentType.

One contract, two algorithms

BookingService will call pay without branching on gateway details. The boolean is a result the orchestrator must handle; calling pay is not the same as confirming a booking.

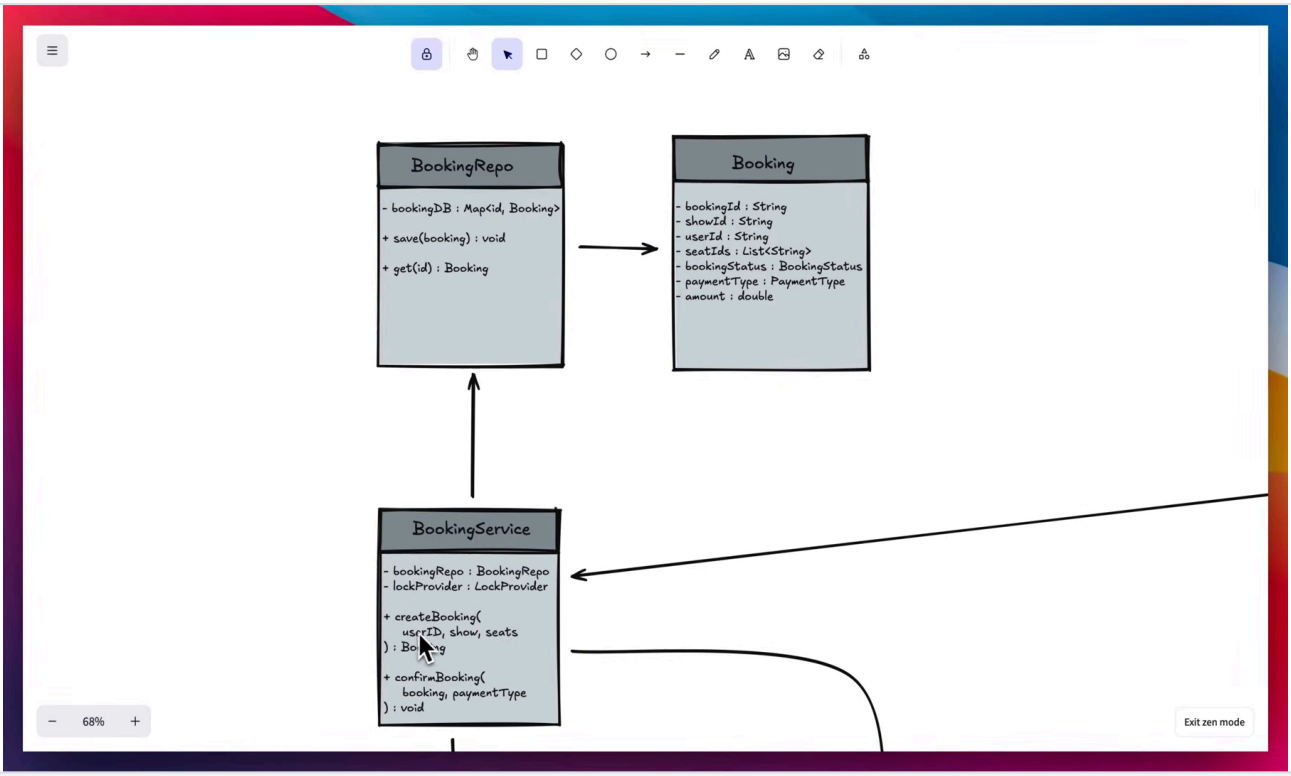
SeatStatus is not enforcement

The video includes AVAILABLE, LOCKED and BOOKED values, but its booking path does not use this enum to preserve permanent seat allocation. The pack retains it as an unused vocabulary type.



11:05 · Whiteboard: PaymentStrategy and the factory. Code names append Strategy to the concrete classes.

Booking state and payment responsibility



15:10 · Booking metadata and its repository on the whiteboard.

State	Meaning / allowed next step in the extension
CREATED	Seats are temporarily held. Confirm, cancel, or fail validation.
CONFIRMED	Payment succeeded and the show-seat allocation was recorded. A confirmation retry is a no-op.
FAILED	Payment returned false, threw a synchronous error, or the hold was invalid when confirmation was attempted.
CANCELLED	The user explicitly cancelled a CREATED attempt. Its holds were released.

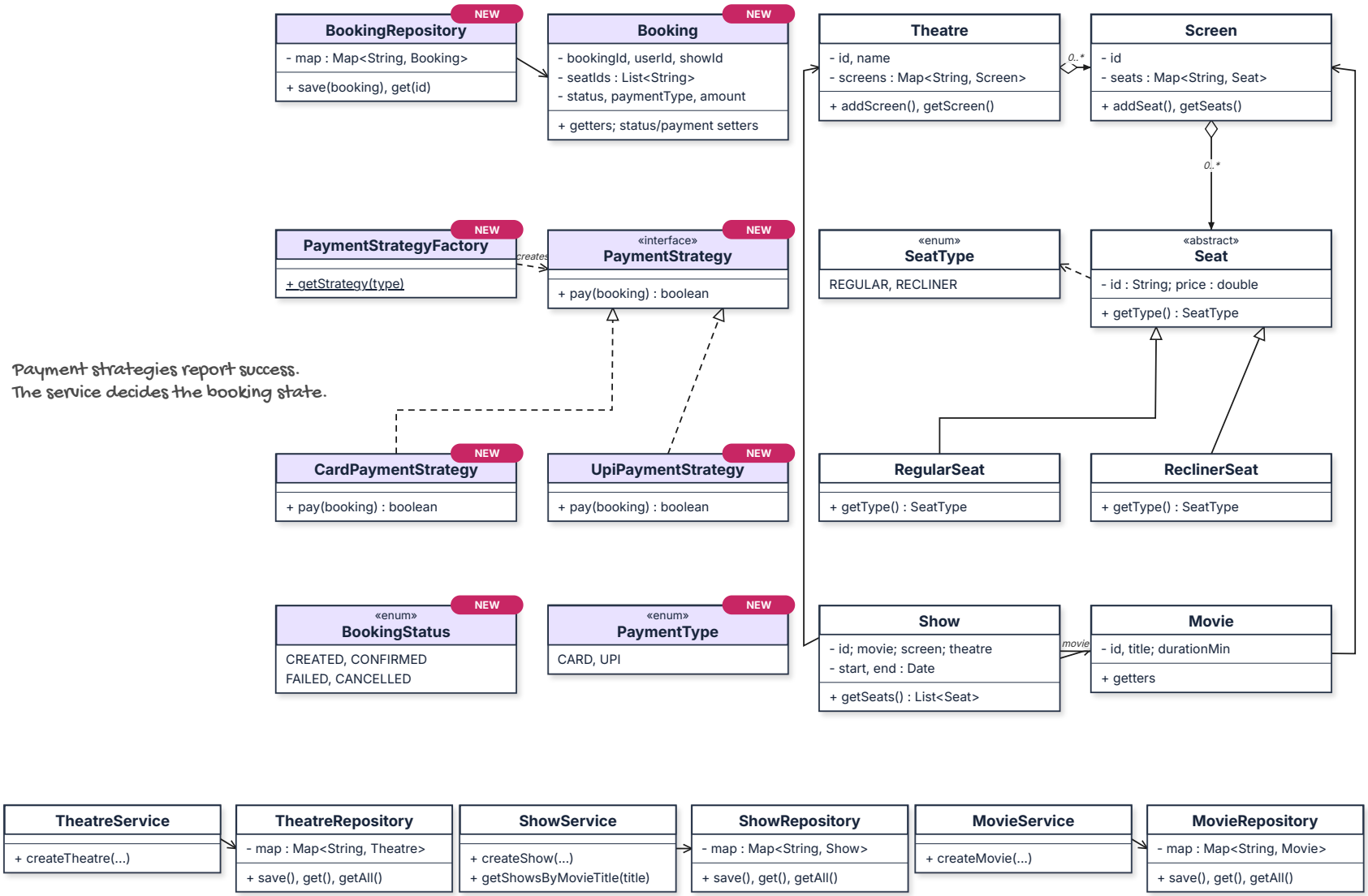
Expiry is not a background status update

A lease can expire while the Booking object still says CREATED. The sweeper removes stale lock entries; it does not update booking rows. Phase 6 turns a stale booking to FAILED when confirmation is attempted. A background booking-expiry workflow is future work.

The Phase 3 output intentionally leaves the booking in CREATED after calling a payment mock directly. There is not yet an orchestration layer to authorize the state transition.

Phase 3 · The growing class diagram

PHASE 3 / GROWING CLASS MAP



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.
Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 3 · Code lab 1

Read from `Phase-3-Bookings-and-Payments/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingStatus.java NEW

```java
public enum BookingStatus { CREATED, CONFIRMED, FAILED, CANCELLED }
```

PaymentType.java NEW

```java
public enum PaymentType { CARD, UPI }
```

SeatStatus.java NEW

```java
public enum SeatStatus { AVAILABLE, LOCKED, BOOKED }
```

PaymentStrategyFactory.java NEW

```java
public class PaymentStrategyFactory {
    public static PaymentStrategy getStrategy(PaymentType type) {
        return switch (type) {
            case CARD -> new CardPaymentStrategy();
            case UPI -> new UpiPaymentStrategy();
        };
    }
}
```

Booking.java NEW

```java
public class Booking {
    private final String bookingId;
    private final String userId;
    private final String showId;
    private final List<String> seatIds;
    private BookingStatus status;
    private PaymentType paymentType;
    private final double amount;
    public Booking(String bookingId, String userId, String showId,
                   List<String> seatIds, BookingStatus status,
                   PaymentType paymentType, double amount) {
        this.bookingId = bookingId;
        this.userId = userId;
        this.showId = showId;
        this.seatIds = List.copyOf(seatIds);
        this.status = status;
        this.paymentType = paymentType;
        this.amount = amount;
    }
    public String getBookingId() { return bookingId; }
    public String getUserId() { return userId; }
    public String getShowId() { return showId; }
    public List<String> getSeatIds() { return seatIds; }
    public BookingStatus getStatus() { return status; }
    public PaymentType getPaymentType() { return paymentType; }
    public double getAmount() { return amount; }
    public void setStatus(BookingStatus status) { this.status = status; }
    public void setPaymentType(PaymentType type) { paymentType = type; }
}
```

BookingRepository.java NEW

```java
public class BookingRepository {
    private final Map<String, Booking> map = new ConcurrentHashMap<>();
    public void save(Booking booking) {
        map.put(booking.getBookingId(), booking);
    }
    public Booking get(String id) { return map.get(id); }
}
```

Phase 3 · Code lab 2

Read from `Phase-3-Bookings-and-Payments/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

CardPaymentStrategy.java — NEW

```
 6 public class CardPaymentStrategy implements PaymentStrategy {
 7     @Override
 8     public boolean pay(Booking booking) {
 9         System.out.printf(Locale.ROOT, "Paid INR %.2f via Card%n",
10             booking.getAmount());
11         return true;
12     }
13 }
```

PaymentStrategy.java — NEW

```
 5 public interface PaymentStrategy {
 6     boolean pay(Booking booking);
 7 }
```

UpiPaymentStrategy.java — NEW

```
 6 public class UpiPaymentStrategy implements PaymentStrategy {
 7     @Override
 8     public boolean pay(Booking booking) {
 9         System.out.printf(Locale.ROOT, "Paid INR %.2f via UPI%n",
10             booking.getAmount());
11         return true;
12     }
13 }
```

Phase 3 · Run it and check yourself

Phase-3-Bookings-and-Payments / actual output

```
Search: interstellar
show1 | Interstellar | PVR Phoenix | s1 | 18:30-20:30 IST
Stored booking: CREATED
Paid INR 450.00 via Card
Payment result: true
Status stays CREATED until orchestration exists.
```

Checkpoint

1. What is the difference between a successful payment and a confirmed booking?
2. Why is this a Simple Factory rather than Factory Method?
3. Which files change to add a new payment method?
4. What information must be persisted for a payment retry?

Predict the dynamic dispatch

Change	Expected observation
Select UPI instead of CARD	The factory returns UpiPaymentStrategy; the same pay call uses the UPI behavior.
Use a regular plus recliner seat	150 + 300 = 450. The payment algorithm should not calculate seat prices.
A mock returns false	The caller receives false. In this phase, no BookingService exists to choose the next state.
Add another PaymentType	The switch must be extended so that the new enum value maps to a strategy.

Retype drill

Implement a temporary declining PaymentStrategy as a lambda. Print its boolean result and the Booking status. Explain why the status does not change automatically and where that decision will belong.

PHASE 4

Phase 4 · Expiring locks

atomic compute · ownership · TTL

Give one owner a temporary claim on one show-seat key, with an expiry deadline.

`Phase-4-Expiring-Locks/`

`InMemoryLockProvider` `LockProvider`

The lease record

One map entry contains a deadline and owner. A key is available when no entry exists or its deadline has passed. The deadline check is part of acquisition; it does not wait for the cleanup thread.

The four operations

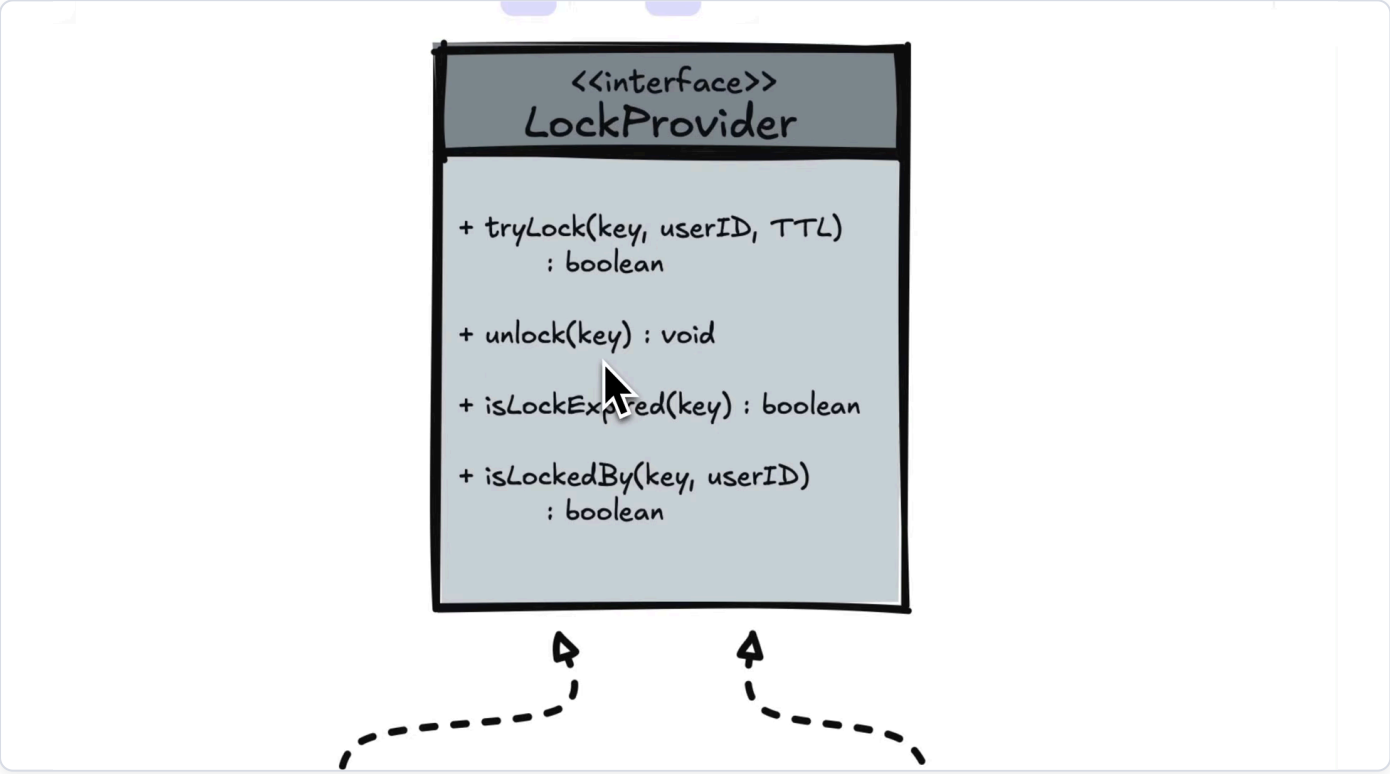
Try to acquire, unlock, check expiry, and check ownership. The default constructor uses system milliseconds. A second constructor accepts a clock for repeatable examples.

Why ConcurrentHashMap.compute?

The read-check-replace decision for a single key must be atomic. Two separate get/put operations could both observe availability. The remapping function runs as one atomic update for that key.

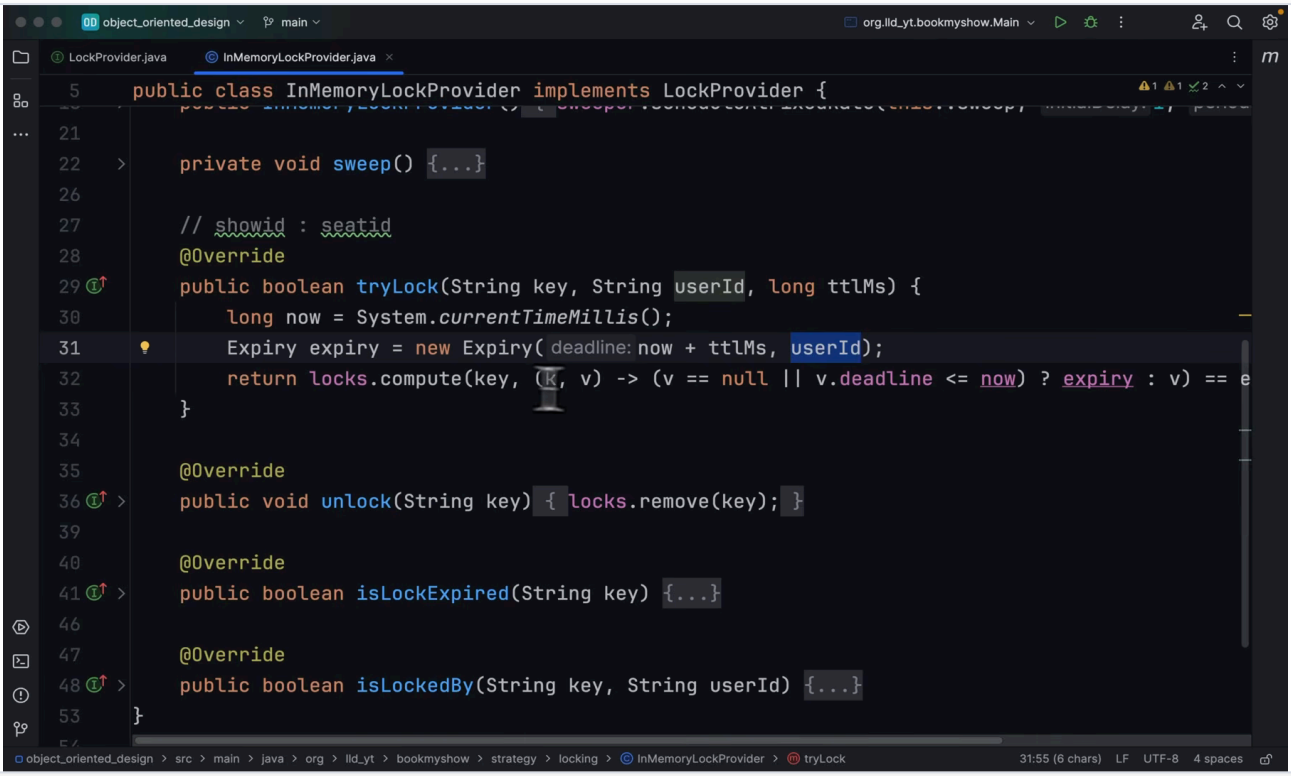
What the sweeper actually does

The minute-based task cleans old entries from memory. It is not the source of truth for validity. The demo can reacquire at five seconds even if the sweeper has not run.



12:40 · The LockProvider interface has four operations.

Walk the atomic acquisition



35:35 · The visible source uses compute and compares the returned object by identity.

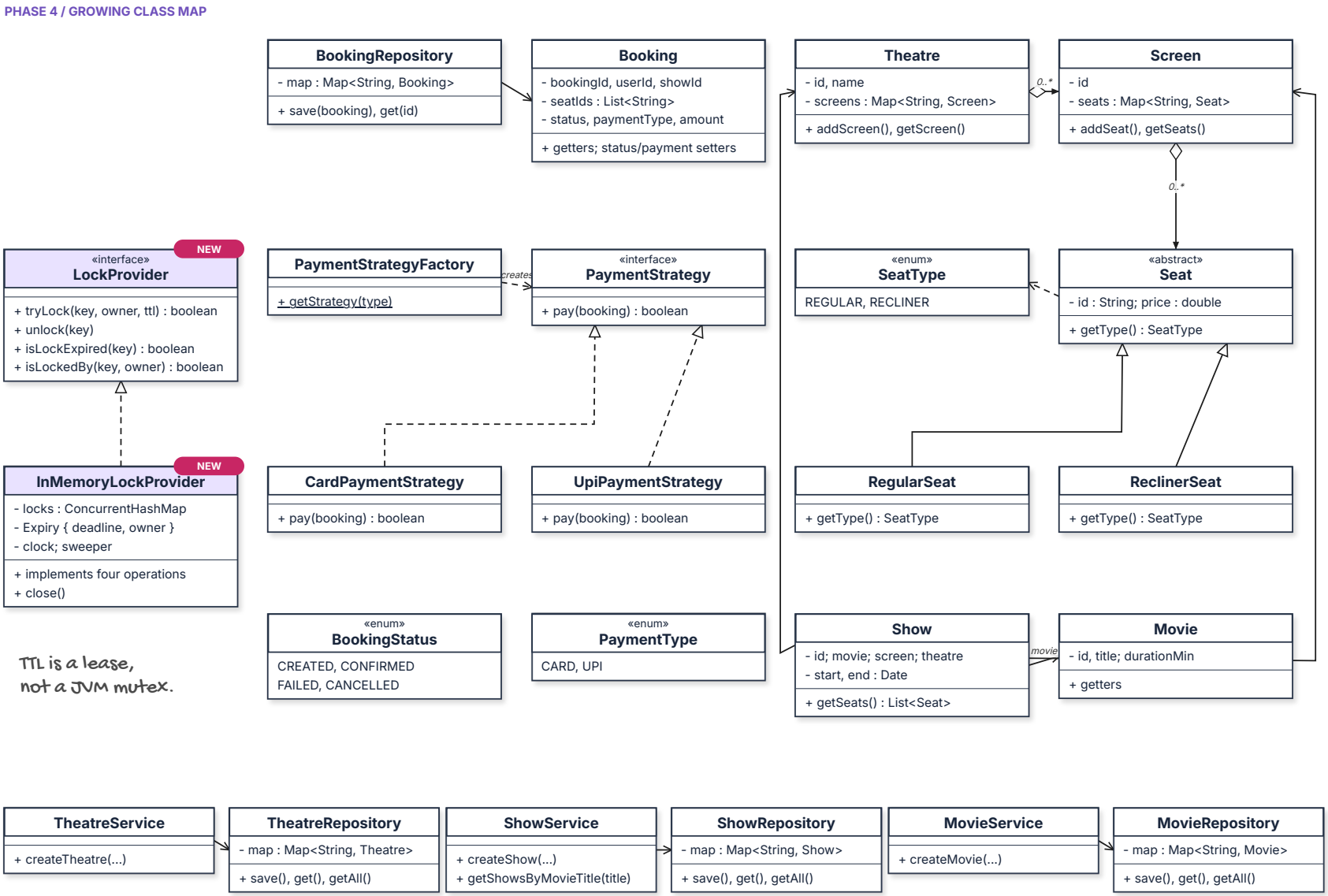
Step	Reason
Create a fresh Expiry(now + ttl, owner)	This object uniquely identifies the candidate acquisition.
compute(key, remapping function)	Inspect and replace the current entry atomically for this key.
If absent or deadline ≤ now, install candidate	An expired entry does not block a new attempt.
Otherwise return the old object	Keep the live owner's claim.
Compare returned object == candidate	True means this call installed its own candidate. It is an identity test, not a comparison of owner strings.

Atomic for one key only

A ConcurrentHashMap does not make a loop over several seats a transaction. It also does not make expiry-check + ownership-check + payment + status-change atomic as a sequence.

Primary API reference: Java 17 ConcurrentHashMap. The long-lived TTL lease is different from a Java monitor lock, which protects a short critical section.

Phase 4 · The growing class diagram



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.
Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 4 · Code lab 1

Read from `Phase-4-Expiring-Locks/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

InMemoryLockProvider.java · 1/2 NEW

```java
public class InMemoryLockProvider implements LockProvider, AutoCloseable {
    private static class Expiry {
        final long deadline;
        final String owner;
        Expiry(long deadline, String owner) {
            this.deadline = deadline;
            this.owner = owner;
        }
    }
    private final ConcurrentHashMap<String, Expiry> locks =
        new ConcurrentHashMap<>();
    private final ScheduledExecutorService sweeper =
        Executors.newSingleThreadScheduledExecutor();
    private final LongSupplier clock;
    public InMemoryLockProvider() { this(System::currentTimeMillis); }
    public InMemoryLockProvider(LongSupplier clock) {
        this.clock = clock;
        sweeper.scheduleAtFixedRate(this::sweep, 1, 1, TimeUnit.MINUTES);
    }
    private void sweep() {
        long now = clock.getAsLong();
        locks.entrySet().removeIf(e -> e.getValue().deadline <= now);
    }
    public boolean tryLock(String key, String userId, long ttlMs) {
        long now = clock.getAsLong();
        Expiry expiry = new Expiry(now + ttlMs, userId);
        return locks.compute(key, (k, old) ->
            old == null || old.deadline <= now ? expiry : old) == expiry;
    }
    public void unlock(String key) { locks.remove(key); }
    public boolean isLockExpired(String key) {
```

InMemoryLockProvider.java · 2/2 NEW

```java
        Expiry expiry = locks.get(key);
        return expiry == null || expiry.deadline <= clock.getAsLong();
    }
    public boolean isLockedBy(String key, String userId) {
        Expiry expiry = locks.get(key);
        return expiry != null && expiry.owner.equals(userId);
    }
    public void close() { sweeper.shutdownNow(); }
}
```

LockProvider.java NEW

```java
public interface LockProvider {
    boolean tryLock(String key, String userId, long ttlMs);
    void unlock(String key);
    boolean isLockExpired(String key);
    boolean isLockedBy(String key, String userId);
}
```

Phase 4 · Run it and check yourself

```
Phase-4-Expiring-Locks / actual output
t=0 user4 acquires: true
t=0 user5 acquires: false
t=5s expired: true
t=5s user5 acquires: true
Old owner still owns: false
Same seat, another show: true
```

Checkpoint

1. Why does using ConcurrentHashMap alone not make get-then-put safe?
2. Why can a new user acquire before the minute-based sweep?
3. What happens exactly at deadline == now?
4. Why is unconditional unlock(key) unsafe after expiry and reacquisition?

Deadline trace

Demo clock	Stored entry	Can user5 acquire?
1,000 ms	user4; deadline = 6,000	No.
5,999 ms	Same live entry	No.
6,000 ms	Entry may still be present, but expired	Yes; compute replaces it.
After replacement	user5; deadline = 11,000	An older attempt cannot claim this is its own lease.

Retype drill

Change the clock increment to 4,999 ms and predict all lines before running. Then restore 5,000 ms. The exact boundary is deadline <= now, not strictly less than now. Close the provider so its scheduler cannot keep the JVM alive.

PHASE

5

Phase 5 · The booking flow

orchestration · concurrency · expiry

Connect selection, locking, payment and confirmation; reproduce the four video demos.

`Phase-5-Booking-Flow/`

`SeatNotAvailableException` `BookingService`

Create the booking

1. Build each show-seat key.
2. Acquire it for the user, or throw.
3. Sum selected seat prices.
4. Store the CREATED booking.

Confirm the booking

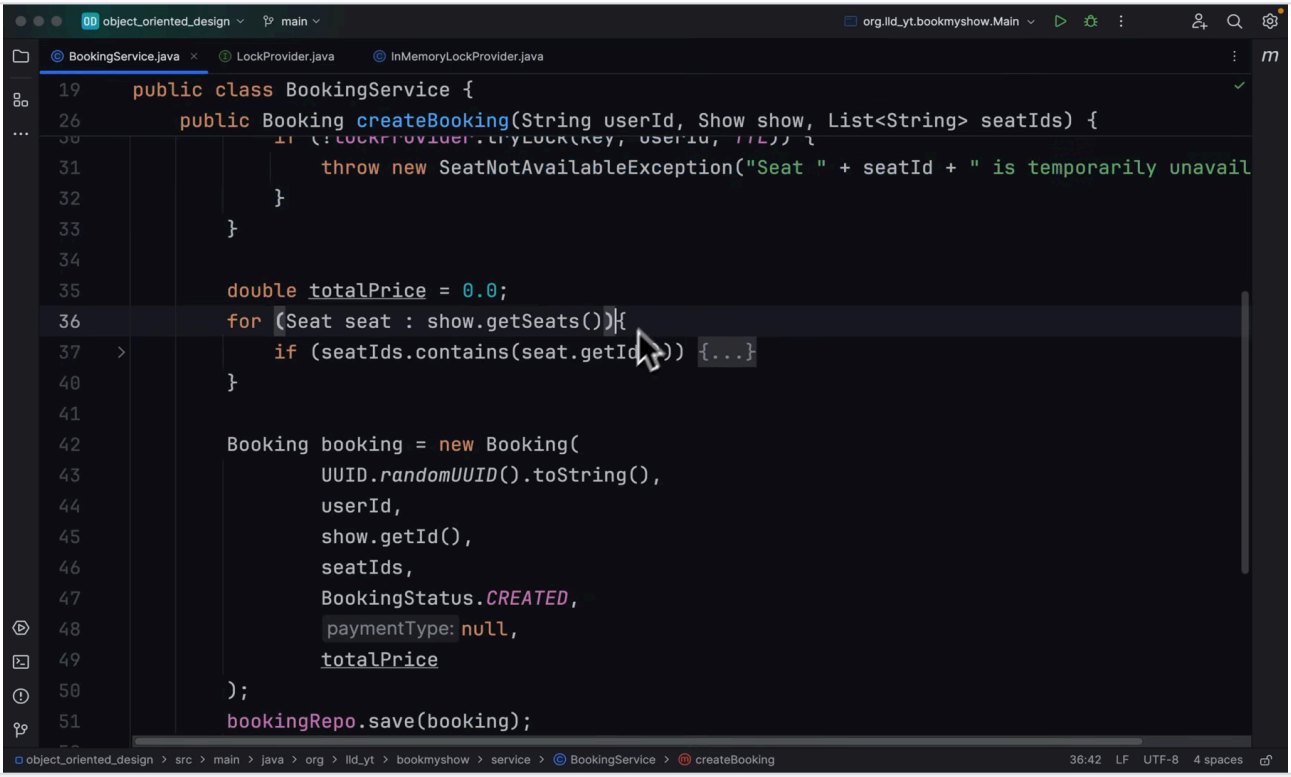
1. Require CREATED.
2. Check expiry and ownership for each seat.
3. Choose a payment strategy and invoke it.
4. Release temporary locks and set CONFIRMED.

Teaching baseline: known gaps

This phase preserves the visible service algorithm. It has no partial rollback, ignores the payment boolean, and does not record permanently sold seats. Do not treat it as the final safe design.

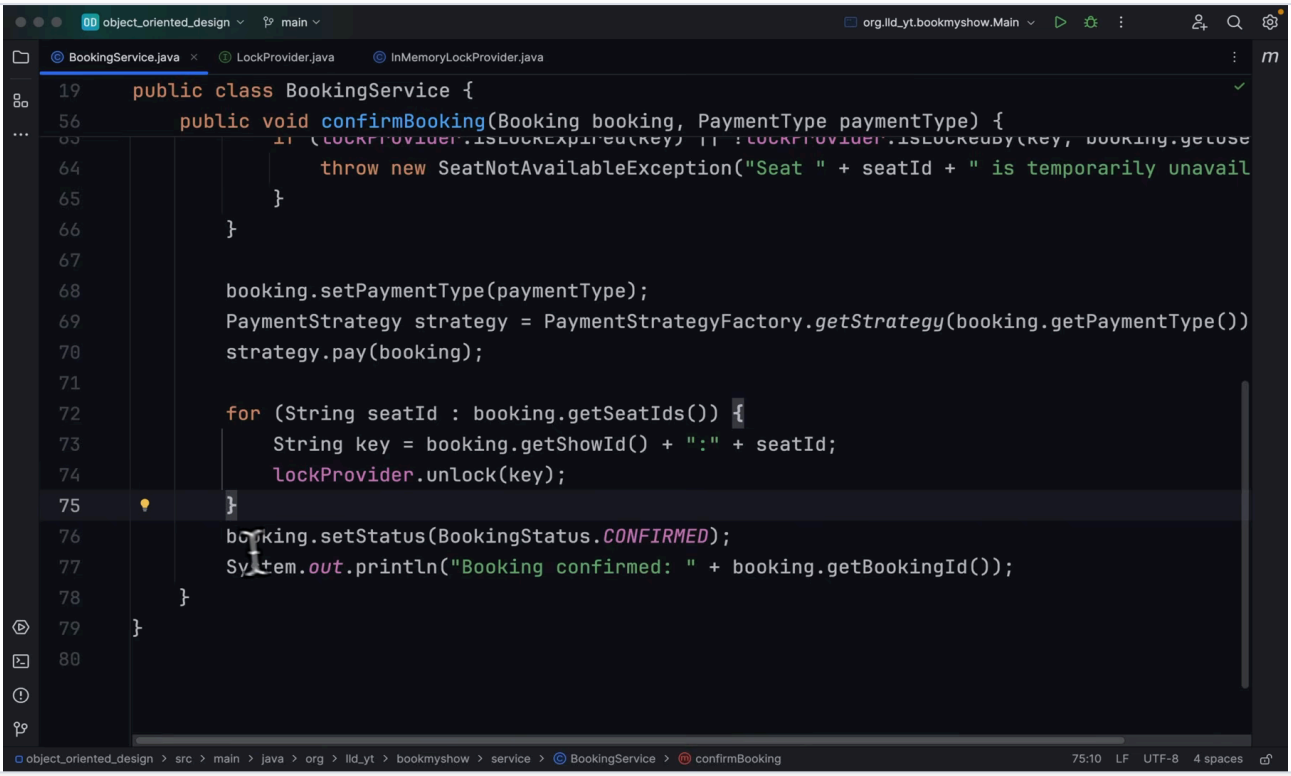
Keep the two actions distinct

Proceed and Pay occur at different times. The hold protects the checkout interval, while confirmation must reject stale attempts. Combining them would hide the main requirement.



39:45 · The source totals matching seats and constructs a CREATED booking.

Read the implementation critically



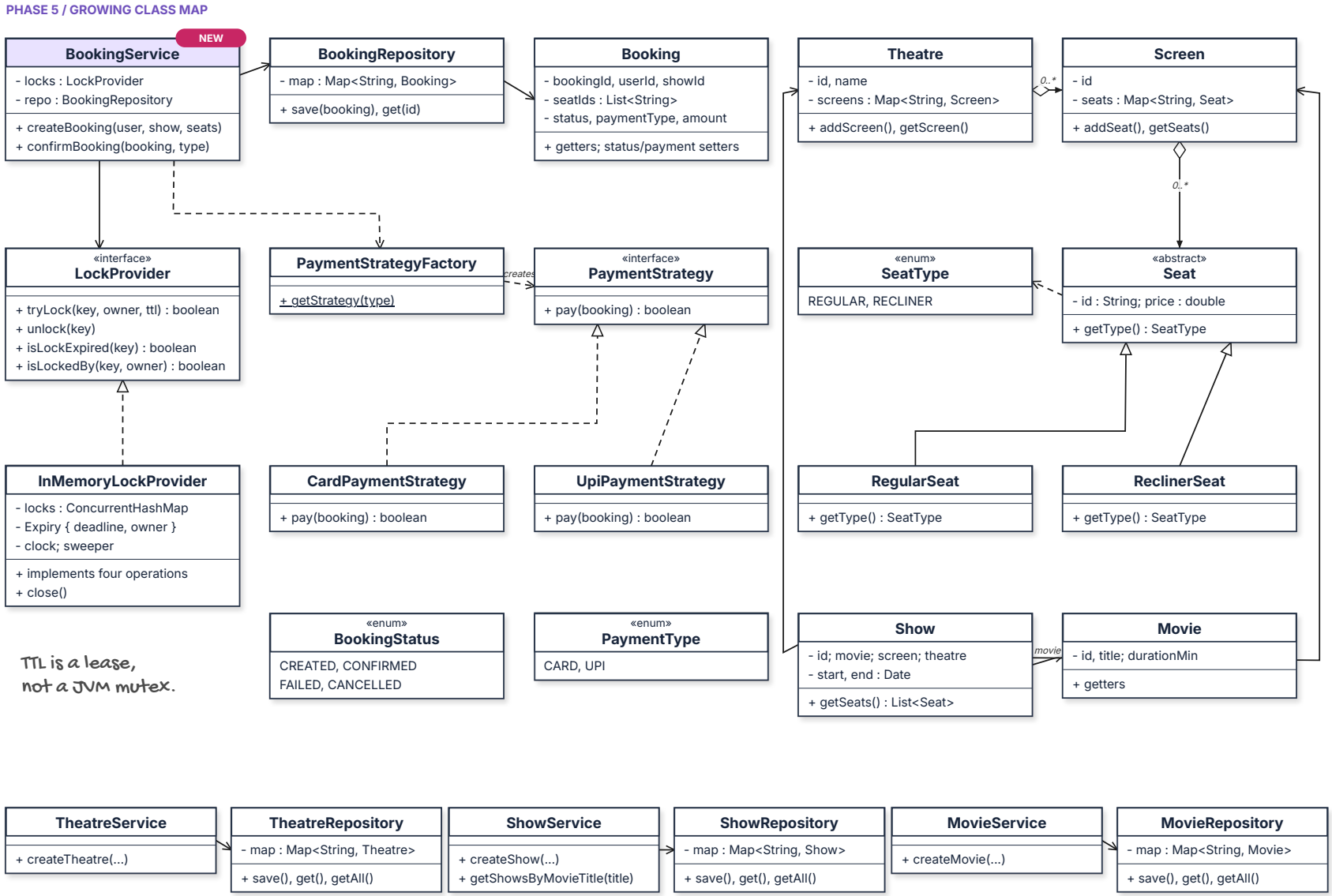
41:30 · pay() is called without checking its result; locks are then removed.

Claim / intent	What the visible code actually establishes
Losing seat groups leave no hold	createBooking throws on a failed seat. It does not undo earlier successful acquisitions.
Seats stay unavailable after payment	confirmBooking removes lease keys and sets Booking status. The shown create path does not consult confirmed bookings.
Failed payment releases without confirmation	The boolean from pay is ignored. Both current mocks always return true, masking this missing branch.
A valid owner can finish safely	Separate checks can become stale before payment finishes. User IDs also do not distinguish two attempts by the same user.

Counterexample to memorize

Let A acquire seat 3 and then fail on seat 4, already held by B. Without rollback, A still blocks seat 3 despite having no booking. A concurrent collection cannot infer the business transaction you intended.

Phase 5 · The growing class diagram



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.
Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 5 · Code lab 1

Read from `Phase-5-Booking-Flow/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

SeatNotAvailableException.java — NEW

```java
public class SeatNotAvailableException extends RuntimeException {
    public SeatNotAvailableException(String message) { super(message); }
}
```

BookingService.java · 1/2 — NEW

```java
public class BookingService {
    private static final long TTL = 5_000;
    private static final AtomicInteger ids = new AtomicInteger();
    private final LockProvider lockProvider;
    private final BookingRepository bookingRepo;
    public BookingService(LockProvider lockProvider,
                          BookingRepository bookingRepo) {
        this.lockProvider = lockProvider;
        this.bookingRepo = bookingRepo;
    }
    public Booking createBooking(String userId, Show show,
                                 List<String> seatIds) {
        for (String seatId : seatIds) {
            String key = show.getId() + ":" + seatId;
            if (!lockProvider.tryLock(key, userId, TTL)) {
                throw new SeatNotAvailableException(
                    "Seat " + seatId + " is temporarily unavailable");
            }
        }
        double totalPrice = 0;
        for (Seat seat : show.getSeats()) {
            if (seatIds.contains(seat.getId())) {
                totalPrice += seat.getPrice();
            }
        }
        Booking booking = new Booking("B" + ids.incrementAndGet(),
            userId, show.getId(), seatIds, BookingStatus.CREATED,
            null, totalPrice);
        bookingRepo.save(booking);
        return booking;
    }
```

Phase 5 · Code lab 2

Read from `Phase-5-Booking-Flow/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingService.java · 2/2 — NEW

```
44      public void confirmBooking(Booking booking, PaymentType paymentType) {
45          if (booking.getStatus() ≠ BookingStatus.CREATED) {
46              throw new IllegalStateException("Booking is not CREATED");
47          }
48          for (String seatId : booking.getSeatIds()) {
49              String key = booking.getShowId() + ":" + seatId;
50              if (lockProvider.isLockExpired(key)
51                  || !lockProvider.isLockedBy(key, booking.getUserId())) {
52                  throw new SeatNotAvailableException(
53                      "Seat " + seatId + " is temporarily unavailable");
54              }
55          }
56          booking.setPaymentType(paymentType);
57          PaymentStrategy strategy =
58              PaymentStrategyFactory.getStrategy(paymentType);
59          strategy.pay(booking);
60          for (String seatId : booking.getSeatIds()) {
61              lockProvider.unlock(booking.getShowId() + ":" + seatId);
62          }
63          booking.setStatus(BookingStatus.CONFIRMED);
64      }
65  }
```

Main.java · 1/2 — CHANGED

```
39          AtomicLong clock = new AtomicLong(1_000);
40          try (InMemoryLockProvider locks = new InMemoryLockProvider(clock::get)) {
41              BookingRepository bookings = new BookingRepository();
42              BookingService service = new BookingService(locks, bookings);
43              System.out.println("Demo 2: user1 selects s1-1 and s1-2");
44              Booking first = service.createBooking("user1", show,
45                  List.of("s1-1", "s1-2"));
46              service.confirmBooking(first, PaymentType.CARD);
47              System.out.println("user1: " + first.getStatus());
48              System.out.println("Demo 3: overlapping [3,4] and [4,5]");
49              ExecutorService pool = Executors.newFixedThreadPool(2);
50              CountDownLatch startGate = new CountDownLatch(1);
51              List<Future<Booking>> results = new ArrayList<>();
52              for (int i = 0; i < 2; i++) {
53                  final int n = i;
54                  results.add(pool.submit(() -> {
55                      startGate.await();
56                      try {
57                          return service.createBooking("user" + (n + 2), show,
58                              n == 0 ? List.of("s1-3", "s1-4")
59                                     : List.of("s1-4", "s1-5"));
60                      } catch (bookmyshow.exception.SeatNotAvailableException e) {
61                          return null;
62                      }
63                  }));
64              }
65              startGate.countDown();
66              int winners = 0;
67              Booking winner = null;
68              for (Future<Booking> result : results) {
69                  Booking candidate = result.get();
```

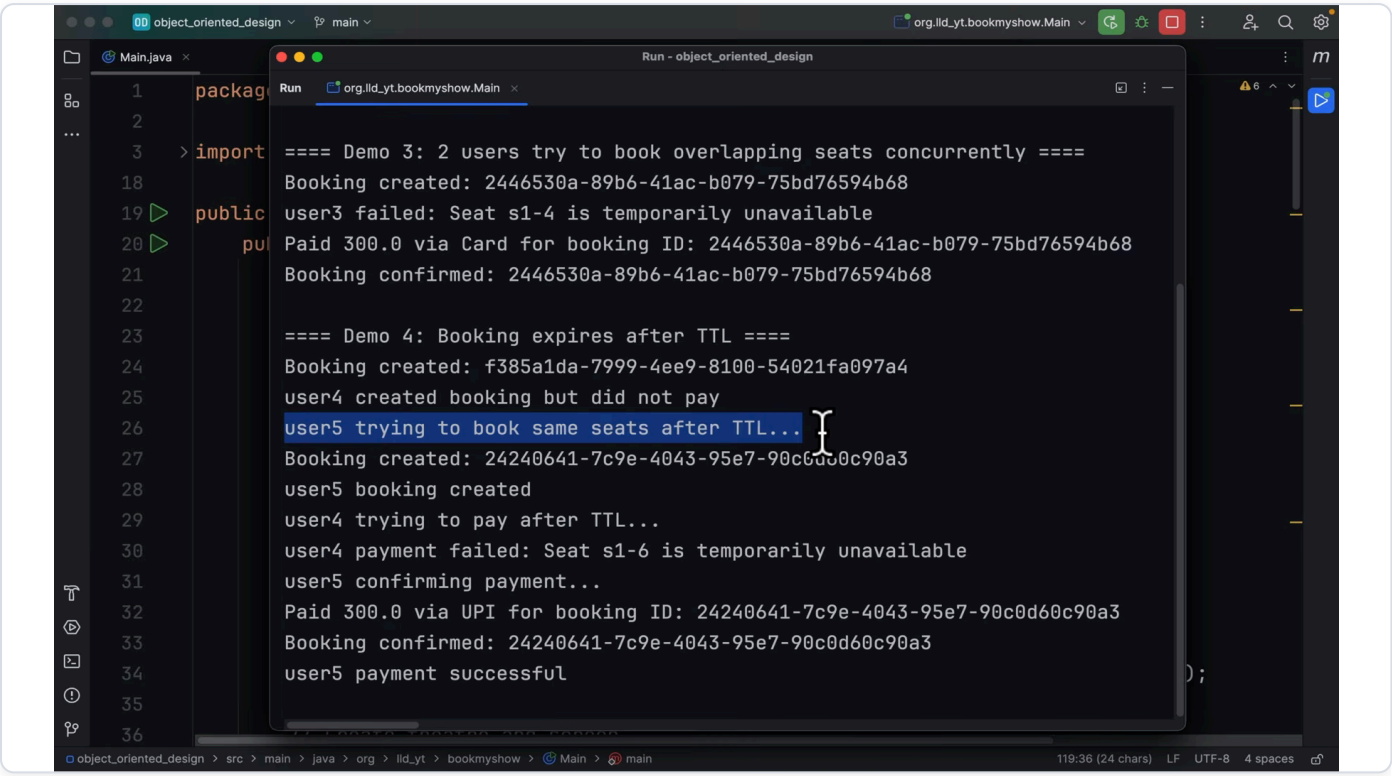
Phase 5 · Code lab 3

Read from `Phase-5-Booking-Flow/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

Main.java · 2/2 CHANGED

```
70                 if (candidate != null) { winners++; winner = candidate; }
71             }
72             pool.shutdown();
73             System.out.println("Winners: " + winners + "; rejected: " + (2 - winners));
74             if (winners != 1) throw new AssertionError("Expected one winner");
75             service.confirmBooking(winner, PaymentType.CARD);
76             System.out.println("Winning booking: " + winner.getStatus());
77             System.out.println("Demo 4: user4 waits 6s with a 5s TTL");
78             Booking old = service.createBooking("user4", show,
79                 List.of("s1-6", "s1-7"));
80             clock.addAndGet(6_000);
81             Booking fresh = service.createBooking("user5", show,
82                 List.of("s1-6", "s1-7"));
83             try {
84                 service.confirmBooking(old, PaymentType.UPI);
85                 throw new AssertionError("Expired booking accepted");
86             } catch (bookmyshow.exception.SeatNotAvailableException e) {
87                 System.out.println("user4 rejected: " + e.getMessage());
88             }
89             service.confirmBooking(fresh, PaymentType.UPI);
90             System.out.println("user5: " + fresh.getStatus());
91         }
92     }
93
```

Compare the four demos with the video



25:00 · The output shows one overlapping request rejected, then user4 losing an expired hold to user5.

What stays the same

Interstellar, PVR Phoenix, ten seats, the overlapping requests [3,4] / [4,5], and user4/user5 after a five-second TTL. Two ₹150 regular seats total ₹300.

Why the output is deterministic

The two requests really run on separate threads. Main waits for both acquisition results before paying, then prints the count of winners rather than the scheduler-dependent winning username.

One video output mismatch

Around 18:55 the shown source selects s1-1 and s1-2, both configured at ₹150; around 23:05 the displayed run reports ₹150. This pack computes ₹300 for the visible two-seat input. The video's later two-seat examples also show ₹300.

Why the clock is injected

Advancing the demo clock by 6,000 ms reproduces expiry without sleeping. It changes the demo harness, not the five-second TTL rule. The default provider constructor still uses real time.

Phase 5 · Run it and check yourself

Phase-5-Booking-Flow / actual output

```
Search: interstellar
show1 | Interstellar | PVR Phoenix | s1 | 18:30-20:30 IST
Demo 2: user1 selects s1-1 and s1-2
Paid INR 300.00 via Card
user1: CONFIRMED
Demo 3: overlapping [3,4] and [4,5]
Winners: 1; rejected: 1
Paid INR 300.00 via Card
Winning booking: CONFIRMED
Demo 4: user4 waits 6s with a 5s TTL
user4 rejected: Seat s1-6 is temporarily unavailable
Paid INR 300.00 via UPI
user5: CONFIRMED
```

Checkpoint

1. Why does the overlapping-seat demo not prove all-or-nothing booking?
2. What would happen if you book s1-1 again after confirmation?
3. Why can the old user's browser still show a Pay button?
4. What would change if a payment strategy returned false?

Challenge the baseline before opening Phase 6

Try to acquire [s1-2, s1-3] while another user holds s1-3, then ask a third user for s1-2. Next, confirm one seat and immediately try to book it again. Write down which invariant each experiment violates. Use fresh fixtures so the earlier demos do not hide the result.

PHASE

6

Phase 6 · Safety extensions

beyond the video · validation · rollback · idempotency

Repair the visible implementation gaps and verify the single-process safety contract.

`Phase-6-Safety-Extensions/`

`UPDATED BookingService` `UPDATED BookingRepository` `UPDATED LockProvider` `UPDATED InMemoryLockProvider` `SafetyChecks`

Beyond the video. This phase strengthens the educational model. It is a single-JVM implementation with synchronous mock payments, in-memory records and a fixed catalog. It does not implement Redis, a payment gateway, durable transactions or crash recovery.

Change the consistency boundary

All booking mutations synchronize on the shared BookingRepository instance. Two BookingService instances using that same repository and lock provider therefore share one boundary. Confirmed seat keys are checked and recorded inside it. The price is serialization: unrelated bookings also wait.

Owner means attempt, not user

The unique booking ID becomes the lease token. A user can abandon one attempt and create another; the old attempt must not validate against or release the new one.

Owner-safe release

unlock(key, token) uses an atomic compare-and-remove. An expired attempt cannot delete a replacement hold owned by a different token.

Validate, then acquire

Reject empty, duplicate and unknown seats before taking locks. If acquisition fails partway, release only keys acquired by this attempt.

Make confirmation repeatable

The service resolves the stored booking, checks caller ownership, and returns early for CONFIRMED. Concurrent retries cannot charge the mock twice within the shared monitor.

The important ordering

Validate the active holds → run the synchronous payment mock → record the sold seat keys → mark CONFIRMED → release temporary holds. A false result or exception marks FAILED and releases this attempt's keys.

What the extension guarantees—and its limits

Situation	Phase 6 behavior
A later seat in a group is unavailable	Earlier acquired keys are rolled back before the failure returns.
The same user creates a fresh attempt after expiry	It has a different booking token. The old attempt fails and cannot remove the new hold.
Payment returns false	Status becomes FAILED; holds are released. No confirmed allocation is written.
A synchronous mock payment throws	Status becomes FAILED, holds are released, and the exception is propagated.
Confirmation is retried or called from two service instances	The shared monitor serializes them. A CONFIRMED retry returns without another charge.
A seat was already confirmed	The repository's bookedSeats set rejects another booking for the same show-seat key.
Cancellation before payment	Only a CREATED booking may become CANCELLED; owner-safe release frees its holds.

A deliberately small proof boundary

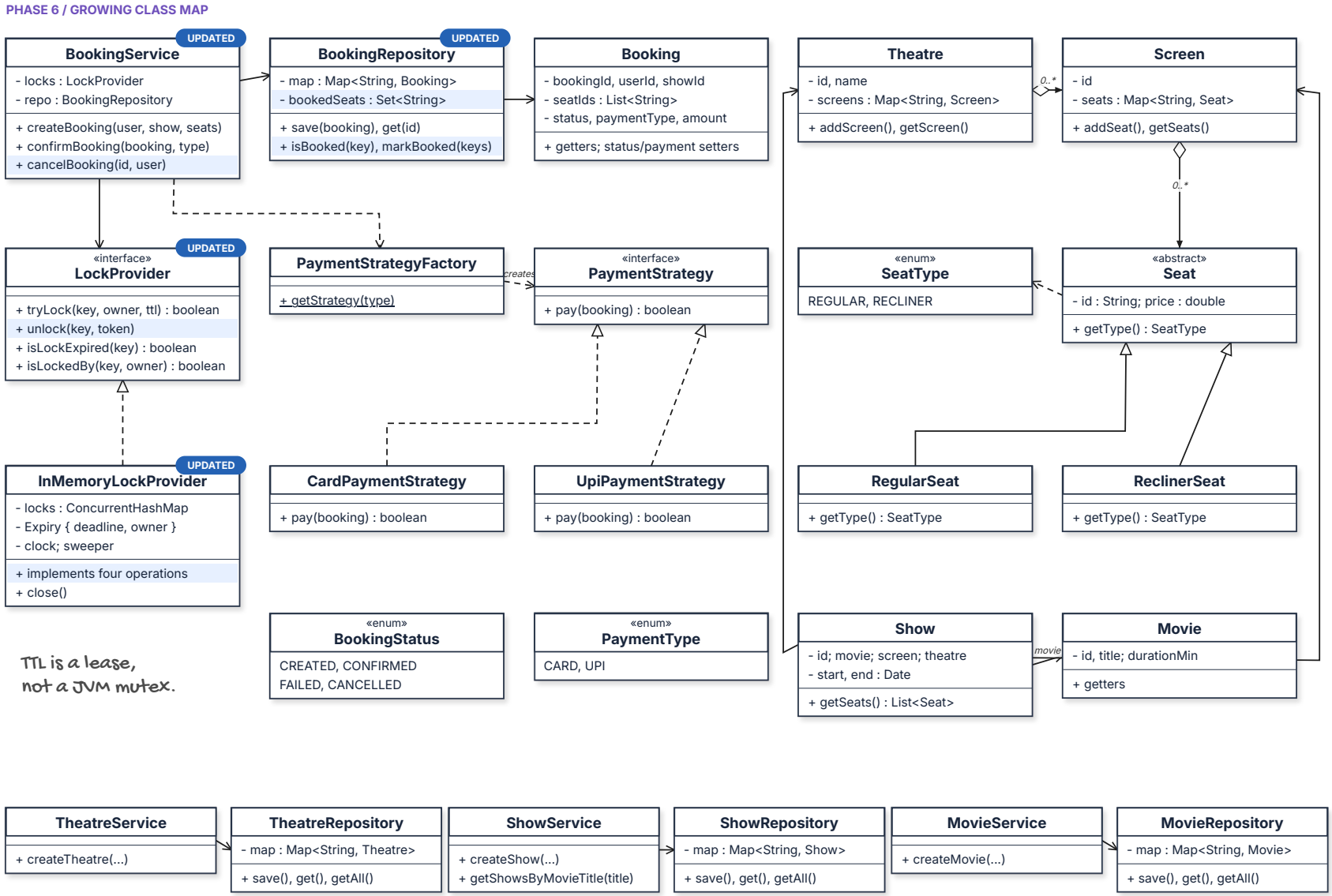
All booking operations must go through services sharing the same repository and lock provider. The catalog is fixed before traffic. In-process objects are trusted; public setters and the compatibility confirm(Booking, type) overload are not an authenticated API. An application boundary should use caller identity and immutable request/response objects.

What if payment takes a long time?

This example keeps the monitor during the synchronous mock call. Once confirmation passes validation, other service calls cannot reacquire its seats while payment runs. Real network payment should not hold this global monitor: use explicit payment-pending state, durable idempotency, transactional allocation, and reconciliation. A timeout can mean an unknown payment outcome, not a definite decline.

Deferred: money remains double for fidelity to the video; use integer minor units or BigDecimal plus currency for real prices. No show-overlap policy, refunds, database persistence, or restart-safe identifiers are implemented.

Phase 6 · The growing class diagram



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.
Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Every class keeps its coordinates across phases. NEW marks additions; UPDATED and blue rows mark extension changes. Open the matching SVG to zoom. Utility entry points and the unused SeatStatus enum are omitted from this business-class map.

Phase 6 · Code lab 1

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

LockProvider.java — CHANGED

```
 3 public interface LockProvider {
 4     boolean tryLock(String key, String ownerToken, long ttlMs);
 5     void unlock(String key, String ownerToken);
 6     boolean isLockExpired(String key);
 7     boolean isLockedBy(String key, String ownerToken);
 8 }
```

InMemoryLockProvider.java · 1/2 — CHANGED

```
 6 public class InMemoryLockProvider implements LockProvider, AutoCloseable {
 7     private static class Expiry {
 8         final long deadline;
 9         final String owner;
10         Expiry(long deadline, String owner) {
11             this.deadline = deadline;
12             this.owner = owner;
13         }
14     }
15     private final ConcurrentHashMap<String, Expiry> locks =
16         new ConcurrentHashMap<>();
17     private final ScheduledExecutorService sweeper =
18         Executors.newSingleThreadScheduledExecutor();
19     private final LongSupplier clock;
20     public InMemoryLockProvider() { this(System::currentTimeMillis); }
21     public InMemoryLockProvider(LongSupplier clock) {
22         this.clock = clock;
23         sweeper.scheduleAtFixedRate(this::sweep, 1, 1, TimeUnit.MINUTES);
24     }
25     private void sweep() {
26         long now = clock.getAsLong();
27         locks.entrySet().removeIf(e -> e.getValue().deadline <= now);
28     }
29     public boolean tryLock(String key, String userId, long ttlMs) {
30         long now = clock.getAsLong();
31         Expiry expiry = new Expiry(now + ttlMs, userId);
32         return locks.compute(key, (k, old) ->
33             old == null || old.deadline <= now ? expiry : old) == expiry;
34     }
35     public void unlock(String key, String ownerToken) {
36         locks.computeIfPresent(key, (k, old) ->
```

InMemoryLockProvider.java · 2/2 — CHANGED

```
37             old.owner.equals(ownerToken) ? null : old);
38     }
39     public boolean isLockExpired(String key) {
40         Expiry expiry = locks.get(key);
41         return expiry == null || expiry.deadline <= clock.getAsLong();
42     }
43     public boolean isLockedBy(String key, String userId) {
44         Expiry expiry = locks.get(key);
45         return expiry != null && expiry.owner.equals(userId)
46             && expiry.deadline > clock.getAsLong();
47     }
48     public void close() { sweeper.shutdownNow(); }
49 }
```

Phase 6 · Code lab 2

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingRepository.java — CHANGED

```
 6 public class BookingRepository {
 7     private final Map<String, Booking> map = new LinkedHashMap<>();
 8     private final Set<String> bookedSeats = new HashSet<>();
 9     public synchronized void save(Booking booking) {
10         map.put(booking.getBookingId(), booking);
11     }
12     public synchronized Booking get(String id) { return map.get(id); }
13     public synchronized boolean isBooked(String key) {
14         return bookedSeats.contains(key);
15     }
16     public synchronized void markBooked(List<String> keys) {
17         bookedSeats.addAll(keys);
18     }
19 }
```

BookingService.java · 1/5 — CHANGED

```
13 public class BookingService {
14     private static final long TTL = 5_000;
15     private static final AtomicInteger ids = new AtomicInteger();
16     private final LockProvider lockProvider;
17     private final BookingRepository bookingRepo;
18     private final java.util.function.Function<PaymentType, PaymentStrategy>
19         payments;
20     public BookingService(LockProvider locks, BookingRepository repo) {
21         this(locks, repo, PaymentStrategyFactory::getStrategy);
22     }
23     public BookingService(LockProvider locks, BookingRepository repo,
24             java.util.function.Function<PaymentType, PaymentStrategy> payments) {
25         this.lockProvider = Objects.requireNonNull(locks);
26         this.bookingRepo = Objects.requireNonNull(repo);
27         this.payments = Objects.requireNonNull(payments);
28     }
29     public Booking createBooking(String userId, Show show,
30                                  List<String> seatIds) {
31         if (userId == null || userId.isBlank() || show == null
32             || seatIds == null || seatIds.isEmpty()) {
33             throw new IllegalArgumentException("User, show and seats required");
34         }
35         List<String> selected = List.copyOf(seatIds);
36         if (new HashSet<>(selected).size() != selected.size()) {
37             throw new IllegalArgumentException("Duplicate seat IDs");
38         }
39         Map<String, Seat> available = new HashMap<>();
40         for (Seat seat : show.getSeats()) available.put(seat.getId(), seat);
41         double amount = 0;
42         List<String> keys = new ArrayList<>();
43         for (String id : selected) {
```

Phase 6 · Code lab 3

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingService.java · 2/5 — CHANGED

```
44              Seat seat = available.get(id);
45              if (seat == null) throw new IllegalArgumentException("Unknown seat");
46              if (!Double.isFinite(seat.getPrice()) || seat.getPrice() < 0) {
47                  throw new IllegalArgumentException("Invalid seat price");
48              }
49              keys.add(key(show.getId(), id));
50              amount += seat.getPrice();
51          }
52          if (!Double.isFinite(amount)) {
53              throw new IllegalArgumentException("Amount overflow");
54          }
55          String token = "B" + ids.incrementAndGet();
56          synchronized (bookingRepo) {
57              List<String> acquired = new ArrayList<>();
58              try {
59                  for (String key : keys) {
60                      if (bookingRepo.isBooked(key)
61                              || !lockProvider.tryLock(key, token, TTL)) {
62                          throw new SeatNotAvailableException("Seat unavailable");
63                      }
64                      acquired.add(key);
65                  }
66                  Booking booking = new Booking(token, userId, show.getId(),
67                          selected, BookingStatus.CREATED, null, amount);
68                  bookingRepo.save(booking);
69                  return booking;
70              } catch (RuntimeException e) {
71                  for (String key : acquired) lockProvider.unlock(key, token);
72                  throw e;
73              }
74          }
```

Phase 6 · Code lab 4

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingService.java · 3/5 — CHANGED

```
75      }
76      public void confirmBooking(Booking booking, PaymentType paymentType) {
77          confirmBooking(booking.getBookingId(), booking.getUserId(), paymentType);
78      }
79      public void confirmBooking(String bookingId, String userId,
80                                 PaymentType paymentType) {
81          Objects.requireNonNull(paymentType, "Payment type required");
82          synchronized (bookingRepo) {
83              Booking booking = requireOwner(bookingId, userId);
84              if (booking.getStatus() == BookingStatus.CONFIRMED) return;
85              if (booking.getStatus() ≠ BookingStatus.CREATED) {
86                  throw new IllegalStateException("Booking is not CREATED");
87              }
88              List<String> keys = keys(booking);
89              for (String key : keys) {
90                  if (bookingRepo.isBooked(key)
91                          || !lockProvider.isLockedBy(key, bookingId)) {
92                      booking.setStatus(BookingStatus.FAILED);
93                      release(booking);
94                      throw new SeatNotAvailableException("Seat unavailable");
95                  }
96              }
97              booking.setPaymentType(paymentType);
98              try {
99                  boolean paid = payments.apply(paymentType).pay(booking);
100                 if (!paid) {
101                     booking.setStatus(BookingStatus.FAILED);
102                     return;
103                 }
104                 bookingRepo.markBooked(keys);
105                 booking.setStatus(BookingStatus.CONFIRMED);
```

Phase 6 · Code lab 5

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

BookingService.java · 4/5 — CHANGED

```
106             } catch (RuntimeException e) {
107                 booking.setStatus(BookingStatus.FAILED);
108                 throw e;
109             } finally {
110                 release(booking);
111             }
112         }
113     }
114     public void cancelBooking(String bookingId, String userId) {
115         synchronized (bookingRepo) {
116             Booking booking = requireOwner(bookingId, userId);
117             if (booking.getStatus() ≠ BookingStatus.CREATED) {
118                 throw new IllegalStateException("Only CREATED can be cancelled");
119             }
120             booking.setStatus(BookingStatus.CANCELLED);
121             release(booking);
122         }
123     }
124     private Booking requireOwner(String id, String userId) {
125         Booking booking = bookingRepo.get(id);
126         if (booking == null || !booking.getUserId().equals(userId)) {
127             throw new IllegalArgumentException("Unknown booking or wrong user");
128         }
129         return booking;
130     }
131     private List<String> keys(Booking booking) {
132         return booking.getSeatIds().stream()
133             .map(id → key(booking.getShowId(), id)).toList();
134     }
135     private void release(Booking booking) {
136         for (String key : keys(booking)) {
```

BookingService.java · 5/5 — CHANGED

```
137             lockProvider.unlock(key, booking.getBookingId());
138         }
139     }
140     private String key(String showId, String seatId) {
141         if (showId == null || showId.isBlank() || showId.contains(":")) {
142             throw new IllegalArgumentException("Invalid show ID");
143         }
144         return showId + ":" + seatId;
145     }
146 }
```

Phase 6 · Code lab 6

Read from `Phase-6-Safety-Extensions/src/bookmyshow`. Purple rows changed since the prior phase. Packages/imports are in the files. Continue split cards in numbered order.

SafetyChecks.java — NEW

```
59              Booking old = service.createBooking("same", show, List.of("s1-4"));
60              time.addAndGet(5_000);
61              Booking fresh = service.createBooking("same", show, List.of("s1-4"));
62              rejected(() -> service.confirmBooking(old, PaymentType.CARD),
63                  "old attempt rejected even for same user");
64              check(locks.isLockedBy("show1:s1-4", fresh.getBookingId()),
65                  "old attempt cannot release the new token");
66              service.confirmBooking(fresh, PaymentType.CARD);
67              BookingService declines = new BookingService(locks, repo,
68                  type -> booking -> false);
69              Booking fail = declines.createBooking("x", show, List.of("s1-5"));
70              declines.confirmBooking(fail, PaymentType.UPI);
71              check(fail.getStatus() == BookingStatus.FAILED,
72                  "payment false produces FAILED");
73              Booking retry = service.createBooking("y", show, List.of("s1-5"));
74              check(retry.getStatus() == BookingStatus.CREATED,
75                  "failed payment frees the seat");
```

Phase 6 · Run it and check yourself

Phase-6-Safety-Extensions / actual output

```
Search: interstellar
show1 | Interstellar | PVR Phoenix | s1 | 18:30-20:30 IST
Demo 2: user1 selects s1-1 and s1-2
Paid INR 300.00 via Card
user1: CONFIRMED
Demo 3: overlapping [3,4] and [4,5]
Winners: 1; rejected: 1
Paid INR 300.00 via Card
Winning booking: CONFIRMED
Demo 4: user4 waits 6s with a 5s TTL
user4 rejected: Seat unavailable
Paid INR 300.00 via UPI
user5: CONFIRMED
Extensions: run safety checks
PASS wrong caller rejected
PASS confirm retry does not recharge
PASS confirmed seat cannot be sold again
PASS empty selection rejected
PASS duplicate selection rejected
PASS unknown seat rejected
PASS overlapping group rejected
PASS partial acquisition rolled back
PASS cancellation releases hold
PASS old attempt rejected even for same user
PASS old attempt cannot release the new token
PASS payment false produces FAILED
PASS failed payment frees the seat
PASS payment exception propagated
PASS payment exception fails booking and releases hold
PASS two service instances charge once
PASS expiry during payment cannot permit a second sale
Safety checks passed: 17
```

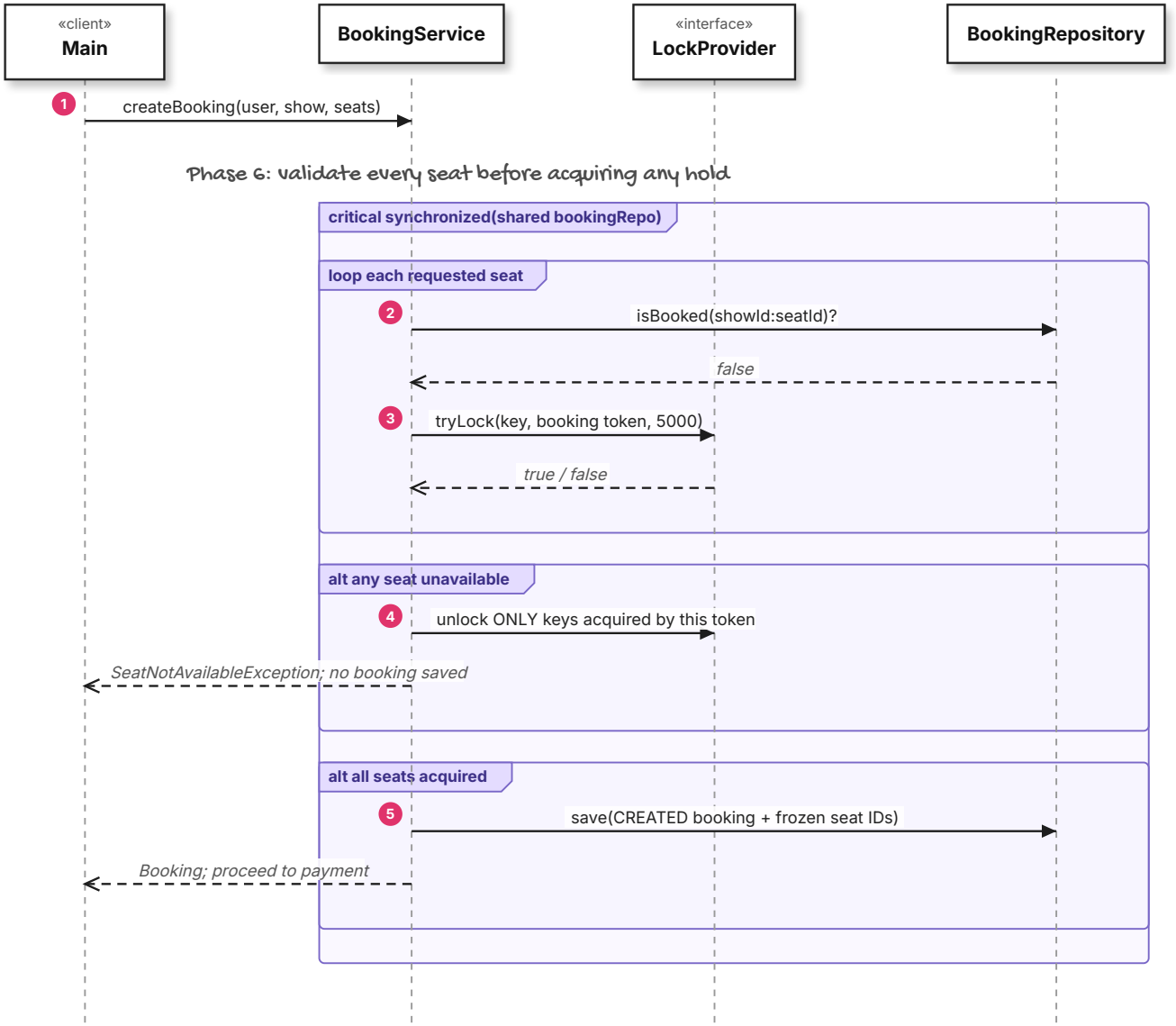
What was verified

The output above contains 17 named assertions. They exercise rejected inputs, stale attempts, rollback, failed/throwing payment mocks, cancellation, confirmed-seat exclusion, concurrent confirmation through two service instances, and expiry during payment. These checks support the stated in-process scope.

Checkpoint

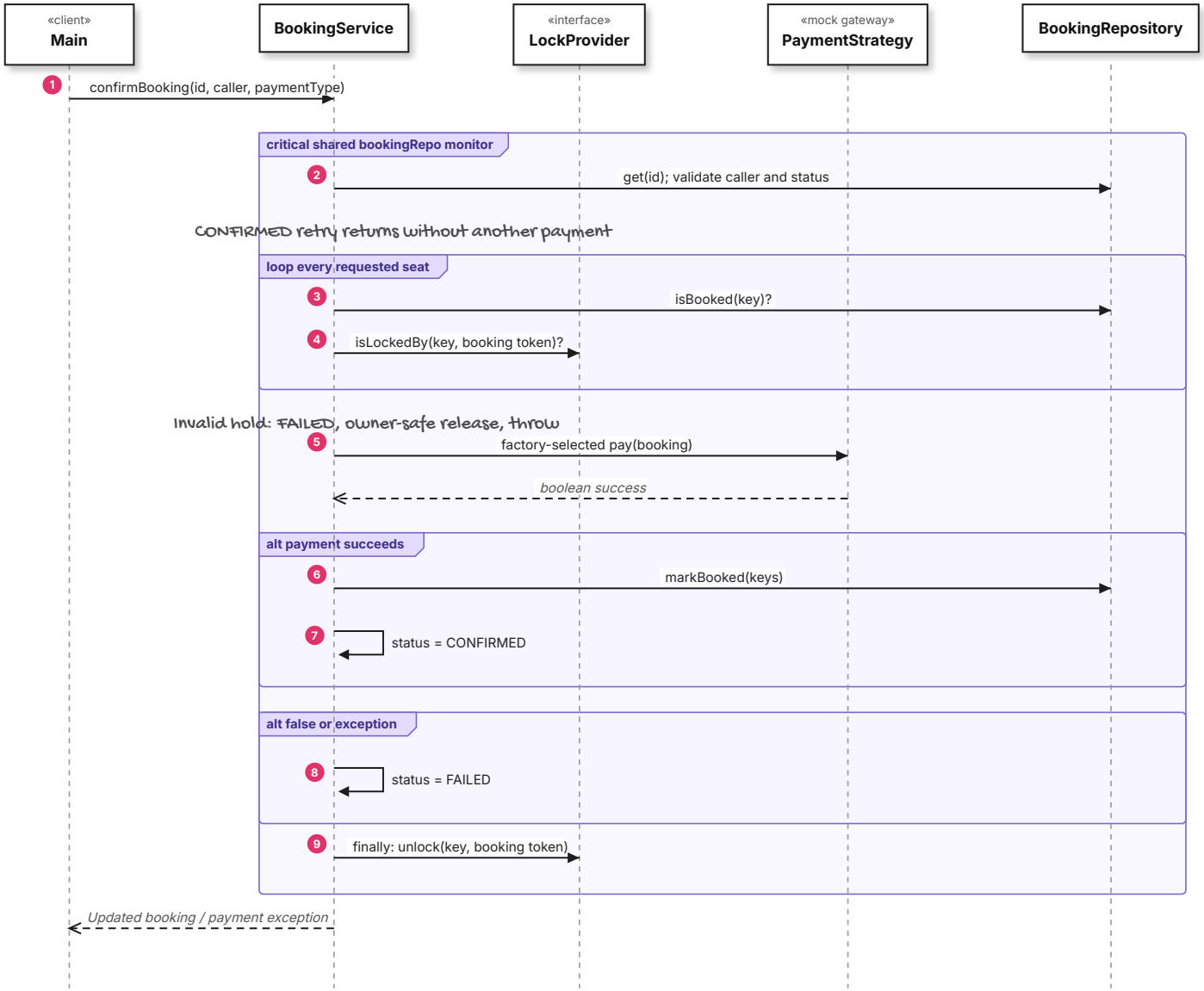
1. Why is userId an insufficient lease token?
2. Why does synchronization belong to the shared repository, not each service instance?
3. Why must permanent allocation be recorded before releasing the lease?
4. What breaks across JVMs or after a restart?

Complete code flow · Create a booking



Phase 6 flow. Main selects a show and seats. The service validates the full request, then checks permanent allocations and acquires all temporary claims inside the shared repository monitor. A failure rolls back only this token's acquired keys. The original Phase 5 has the acquisition loop but lacks rollback and the permanent-booking check.

Complete code flow · Confirm a booking



Phase 6 flow. Caller and booking state are checked first. All seat holds must still belong to this attempt. Payment is a synchronous mock inside the monitor. Success records permanent allocations before releasing holds; failure releases without selling. A confirmed retry returns without charging again.

Explain it in 30 seconds

> **“The catalog tells me which show a seat belongs to. The booking service decides who may claim it.”**

“I separate Movie, Show, Theatre, Screen and Seat. On Proceed, BookingService tries to hold every selected show-seat key for a short TTL. A lease records both its owner token and deadline. On Pay, the service validates the booking and its active holds, selects a PaymentStrategy through a factory, and confirms only after payment succeeds.

“The extension rolls back partial acquisition, uses booking tokens for safe release, remembers permanently booked seats and makes confirmation retries idempotent within one JVM. Across servers, I need shared coordination and durable allocation, plus payment idempotency and recovery.”

Trace the expiry scenario aloud

Moment	show1:s1-6
t = 0	owner = booking A; deadline = 5 seconds
t = 6	A is expired; booking B acquires a new lease
A tries to confirm	A’s token is no longer valid. Do not invoke payment.
A’s failure cleanup	Compare token; do not delete B’s lease.
B confirms	Record sold seat, then remove B’s lease. Further create calls reject the sold seat.

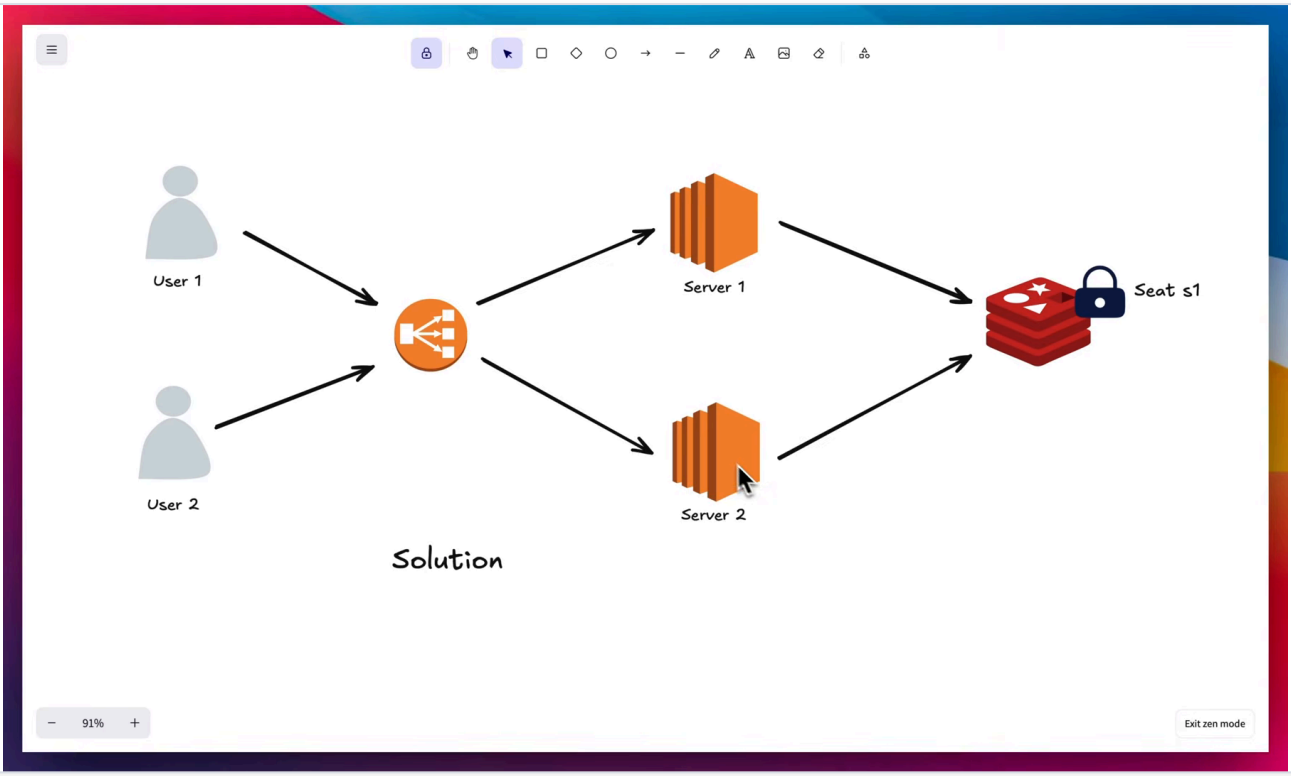
Cost model

In the teaching baseline, acquiring K seats uses K map operations. Pricing scans N screen seats and calls List.contains over K IDs, giving O(NK) work. Phase 6 builds an ID map once and validates K selections, giving expected O(N + K) work before acquisition. Sweeping visits the current L lock entries, O(L). These are algorithmic estimates, not performance measurements.

The question after the diagram

“Where is the atomic boundary?” Answer separately for per-key lease acquisition, multi-seat booking, confirmation, and cross-server persistence. Naming ConcurrentHashMap or Redis is not enough.

From one server to several



44:50 · The presenter replaces separate server-local locks with a shared Redis lock provider.

Two servers with separate maps can each grant the same show-seat key. Shared coordination is required. Redis is one option; a database transaction with suitable uniqueness and concurrency control is another. The title's "Redis is must" should be understood as the need to coordinate shared inventory, not as proof that Redis is the only correct choice.

Redis sketch: acquisition and release

A lease can be created with `SET key unique-token NX PX ttl`. Release must atomically compare the stored token and delete only on a match. Plain DEL is unsafe after expiry and reacquisition. Consult the official documentation for failure and timing assumptions.

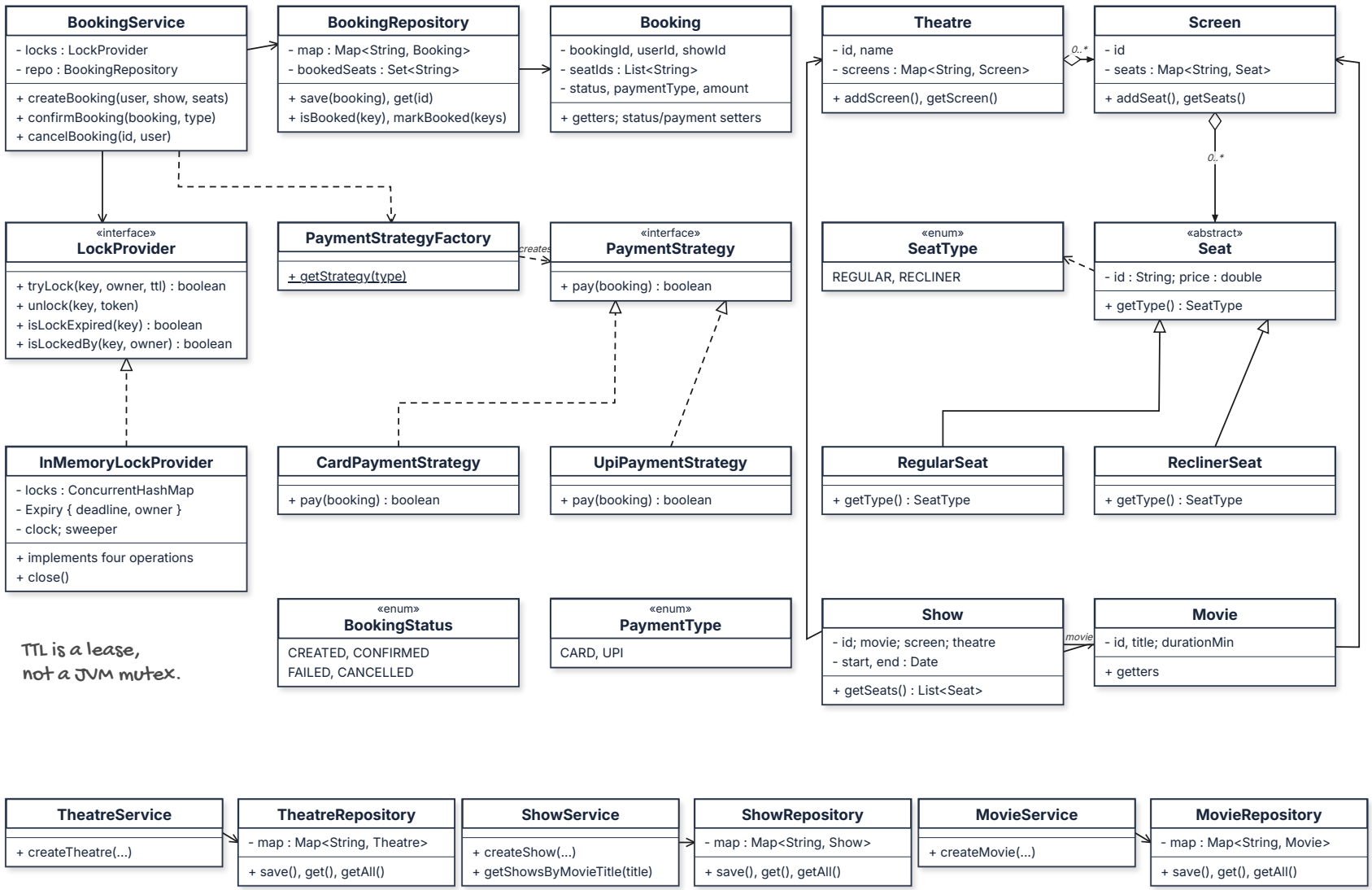
Primary reference: Redis distributed-lock documentation. No Redis code is shipped or exercised in this pack.

Shared locks are not the whole booking transaction

The persistent store must prevent duplicate confirmed allocation for (show, seat). A gateway charge and a database update can fail independently, so production design also needs durable idempotency and reconciliation or compensation. The in-memory extension cannot survive process loss.

The complete design

COMPLETE DESIGN / PHASE 6



Selected members; getters and constructors omitted. Main, SafetyChecks, the exception and unused SeatStatus enum are outside this business-class map.

Diamonds: shared whole/part ownership. Hollow triangles: inheritance or interface implementation. Dashed open arrows: dependency.

Final extension, without change tags. The top-left service coordinates bookings, leases and payments; physical venue entities remain on the right. The lower strip shows catalog services and repositories. The two sequence diagrams specify the operation order that a class diagram cannot express.

Notes on the reference code

What was available

The public repository was found and downloaded, but its current main branch has no matching BookMyShow package. The video description promises code later. This study implementation is therefore a reconstruction from the complete transcript and readable code frames, not an official source export.

Whiteboard / video	Pack decision
Theatre in source; sometimes “theater” in narration	Use the source spelling Theatre, TheatreRepository and TheatreService.
BookingRepo on the board; BookingRepository in source	Use BookingRepository.
CardPayment / UpiPayment on the board	Use the visible source names CardPaymentStrategy and UpiPaymentStrategy.
RedisLockProvider drawn, not demonstrated	Discuss the distributed design; do not pretend a placeholder is a working Redis adapter.
SeatStatus declared but not wired into the booking algorithm	Keep the enum unused and explain why it does not prevent a second sale.
Narration says failed groups leave no partial lock	The shown createBooking loop throws without rollback. Phase 6 explicitly adds rollback.
Narration says confirmed seats are permanently unavailable	The shown service removes locks without checking a sold-seat store. Phase 6 adds bookedSeats.
pay() returns boolean; result ignored in confirmation	Phase 5 preserves the visible call; Phase 6 handles false and exceptions.

Source landmarks

25:56 · Code walkthrough · 33:30 · LockProvider and compute · 37:48 · BookingService · 41:47 · Multiple servers and Redis

Adaptations, assumptions and next exercises

Adaptation	Why it exists
Plain Java 17 and the bookmyshow package	Lombok constructors/getters are written explicitly; no build tool is needed.
LinkedHashMap for catalog output; ConcurrentHashMap for booking storage in the baseline	Stable catalog iteration and safe individual repository map operations during the threaded demo.
Date and seat-ID defensive copies	Keep the selected collection and times stable after construction. The models remain teaching objects, not immutable API DTOs.
Sequential in-process booking IDs instead of random UUIDs	Readable local attempts; uniqueness is scoped to a JVM run. A real deployment needs durable/global IDs.
Injected LongSupplier clock and AutoCloseable provider	Reproducible TTL demonstrations and orderly shutdown of the sweeper thread.
Threaded demo prints winner count; payment waits until both acquisition attempts complete	Preserve a real overlap while keeping output deterministic. The baseline's early unlock would otherwise let the later thread acquire after payment.
Fixed date/time and corrected two-seat price	The pack prints 18:30–20:30 IST and ₹300 for the displayed pair of regular seats.
Phase 6 changes lock ownership, release API, repository and service	These are safety extensions, clearly separated from video reconstruction.

Build these next

1. Replace double with minor units and an explicit currency.
2. Model durable show-seat inventory with a unique confirmed allocation.
3. Introduce payment-pending state and an idempotent gateway adapter. Handle ambiguous timeouts and crash recovery.
4. Implement a real shared lock adapter and test ownership-safe release under expiry.
5. Add show-overlap validation, refunds, catalog validation and immutable application-facing objects.

Public repository snapshot: fd753ebd1fc03b6a499c8671151f62ed7dc13c05. Source and timestamp inventory: `SOURCES.md`.

Self-test · Interview questions

Cover the answer cues, explain your answer aloud, then check. Draw the design from a blank page before looking at the complete map.

1. **Why must the lock key contain the show ID?**
   One physical seat participates in many screenings; each screening has independent inventory.
2. **Why use a booking token instead of user ID?**
   The same user can have an old expired attempt and a fresh attempt. Only the fresh token should authorize confirmation or release.
3. **What does compute make atomic?**
   The map update for one key. It does not atomically acquire several keys or wrap payment plus database writes.
4. **Why is the sweeper not the expiry mechanism?**
   Deadline comparisons determine validity at acquisition and validation time. The sweeper only reclaims entries.
5. **How can a losing request block a seat?**
   It may acquire an earlier seat before a later acquisition fails. Roll back exactly the keys this attempt acquired.
6. **Why is unconditional unlock unsafe?**
   After a lease expires, another attempt may own the key. Deleting by key alone can erase the replacement.
7. **Why can two confirmations charge twice?**
   Both may observe CREATED before either updates it. Serialize the whole decision or use an equivalent transactional state transition and payment idempotency.
8. **What prevents resale after successful payment?**
   A permanent per-show allocation checked during create, not merely the absence or presence of a temporary lease.
9. **What happens when pay returns false?**
   The extension sets FAILED, leaves no confirmed allocation and releases its own temporary holds.
10. **Why can a payment timeout be different from false?**
    The gateway may have charged even though the response was lost. Reconcile using an idempotency key and durable payment state.
11. **Why is synchronizing on each service instance insufficient?**
    Two services can share inventory but have different monitors. The example uses the shared repository as its monitor.
12. **Does Redis alone make the booking correct?**
    No. Shared coordination, durable allocation and payment recovery are separate concerns.
13. **How do you add another payment method?**
    Add an enum value and a strategy, then extend the factory mapping. Booking orchestration continues calling the interface.
14. **Which tests challenge the design most directly?**
    Overlapping groups, expiry and reacquisition, same-user stale attempts, failed payment, repeated confirmation and post-confirmation resale.

Final drill: build Phase 5, expose its gaps, then explain every change in Phase 6.